

PlanifyApp:

Aplicación web de Gestión de Horarios y Personal para PYMEs

Memoria Trabajo Final de Máster

UOC

Pilar López Fernández

**Máster en Desarrollo de
Aplicaciones y Sitios Web**

Nombre Tutor/a de TF
Carles Arnal Castello

**Profesor/a responsable de la
asignatura**
César Córcoles

Fecha Entrega

**Universitat Oberta
de Catalunya**



Esta obra está sujeta a una licencia de
Reconocimiento-NoComercial-
SinObraDerivada [3.0 España de Creative
Commons](https://creativecommons.org/licenses/by-nc-nd/3.0/es/)

FICHA DEL TRABAJO FINAL

Título del trabajo:	<i>PlanifyApp: aplicación web de gestión de horarios y personal para PYMEs</i>
Nombre del autor:	<i>Pilar López Fernández</i>
Nombre del consultor/a:	<i>Carles Arnal Castello</i>
Nombre del PRA:	<i>César Córcoles</i>
Fecha de entrega (mm/aaaa):	<i>06/2026</i>
Titulación o programa:	Máster en Desarrollo de Aplicaciones y Sitios Web
Área del Trabajo Final:	<i>Desarrollo de Aplicaciones Web</i>
Idioma del trabajo:	<i>Catalán, castellano o inglés</i>
Palabras clave	<i>Planificación de turnos, gestión de recursos humanos, desarrollo de aplicaciones web</i>

Resumen del Trabajo

Este trabajo describe el diseño y desarrollo de una aplicación web orientada a la gestión de horarios y personal para pequeñas y medianas empresas (PYMEs). La solución responde a la carencia actual de herramientas económicas para la gestión y organización de plantillas medianas o reducidas, con turnos rotativos y variedad de funciones entre los empleados.

Esta aplicación permite a los administradores modificar horarios semanales y/o mensuales, asignar turnos atendiendo a las aptitudes, preferencias y disponibilidad de los empleados y gestionar sus fichajes e incidencias. Además, cada empleado contará con un portal propio para consultar y planificar sus turnos, recibir notificaciones y realizar solicitudes.

El proyecto se desarrollará con tecnologías modernas como Angular (frontend), Node.JS (backend) y bases de datos relacionales. Se aplicarán principios de diseños responsive y de accesibilidad web. El resultado final será una aplicación funcional y accesible desplegada en un servidor público.

Abstract

This project presents the design and development of a web application aimed at supporting staff scheduling and workforce management for small and medium-sized enterprises (SMEs). The solution addresses the current lack of affordable tools tailored to organizations with small or medium-sized teams, especially those operating with rotating shifts and diverse employee roles.

The application enables administrators to modify weekly and monthly schedules, assign shifts based on employees' skills, preferences, and availability, and manage time tracking and incident reporting. Additionally, each employee has access to a personal portal where they can view and plan their shifts, receive notifications, and submit requests.

The project is developed using modern technologies such as Angular for the frontend, Node.js for the backend, and relational databases. It incorporates responsive design principles and web accessibility standards. The final outcome is a functional and accessible application deployed on a public server.

Índice

1. Introducción	1
1.1. Contexto y justificación del Trabajo	1
1.2. Objetivos del Trabajo	2
1.3. Impacto en sostenibilidad, ético-social y de diversidad	4
1.3.1. Sostenibilidad	4
1.3.2. Comportamiento ético y responsabilidad social	4
1.3.3. Diversidad, género y derechos humanos	5
1.4. Enfoque y método seguido	7
1.5. Planificación del Trabajo	9
2. Material y métodos	2
2.1. Definición de diagramas y casos de uso	2
2.2. Estudio de usabilidad	1
2.2.1. Evaluación heurística	1
2.2.2. Prueba de usabilidad con usuarios reales	3
2.2.3. Principios de accesibilidad (WCAG 2.2 nivel AA)	4
2.3. Wireframes, diseño inicial y diagramas de flujo	5
2.3.1. Wireframes y prototipos en Figma	5
2.3.2. Diagramas de flujo de navegación	6
2.4. Estudio de arquitectura de la aplicación	7
2.4.1. Patrón arquitectónico: cliente-servidor de tres capas	8
2.4.2. Modelo de datos y API REST	8
2.5. Benchmarking, análisis de competidores y público objetivo	11
2.5.1. Análisis de mercado	11
2.5.2. Competidores directos	12
2.5.3. Análisis DAFO	14
2.5.4. Público objetivo y User Persona	15
2.6. Estudio de Viabilidad	16
2.6.1. Viabilidad técnica	16
2.6.2. Viabilidad económica	17

2.6.3.	Viabilidad de mercado	17
2.7.	Desarrollo de la aplicación.....	18
2.7.1.	Entorno de desarrollo	18
2.7.2.	PostgreSQL.	19
2.7.3.	Desarrollo back end con Nojde.js, Express y TypeScript.	22
2.7.4.	Desarrollo front-end con angular.....	26
2.7.5.	Test de validación	31
2.7.6.	Despliegue en servidor público.	33
3.	Resultados	1
3.1.	Estudio de usabilidad	1
3.1.1.	Pruebas de evaluación heurística	1
3.1.2.	Pruebas con usuarios reales	2
3.2.	Desarrollo de la aplicación.....	1
3.2.1.	Test unitarios.....	2
3.2.2.	Despliegue en servidor público	2
3.2.3.	Aspectos pendientes	3
4.	Bibliografía	4

Lista de Tablas

Tabla 1. Orden de hitos y planificación del trabajo.....	9
Tabla 2. Actores principales de casos de uso.	2
Tabla 3. Descripción detallada de casos de uso prioritarios.	1
Tabla 4. Los 10 heurísticos de Nielsen y su aplicación en PlanifyApp.	2
Tabla 5. Tareas que realizar en la ejecución de la prueba.....	3
Tabla 6. Entidades principales del modelo de datos.	8
Tabla 7. Endpoints de la API REST para el MVP de PlanifyApp.....	9
Tabla 8. Evaluación de competidores directos.	12
Tabla 9. Análisis DAFO de Planify.	14
Tabla 10. User persona 1: rol administrador.	15
Tabla 11. User persona 2: rol empleado.	15
Tabla 12. Herramientas utilizadas para el entorno de desarrollo de PlanifyApp	18
Tabla 13. Estructura de carpetas del backend.	23
Tabla 14. Estructura de carpetas del frontend.....	27
Tabla 15. Resultados de la prueba de evaluación heurística.	1
Tabla 16. Resultados de la prueba de usabilidad con usuarios.	2

Lista de Ilustraciones

Ilustración 1. Leyenda de los ODS (Objetivos de Desarrollo Sostenible) en los que este TFM puede contribuir positivamente.....	4
Ilustración 2. Diagrama de Gantt para la planificación del proyecto. Elaboración propia.	1
Ilustración 3. Planificación del proyecto con predicción de riesgos y plan de contingencia.	1
Ilustración 4. Diagrama de casos de uso siguiendo la nomenclatura UML. Elaboración propia.	3
Ilustración 5. Diagrama de flujo. Elaboración propia.	7
Ilustración 6. Diagrama Entidad-Relación de las principales entidades del modelo de datos de PlanifyApp.	10

1. Introducción

1.1. Contexto y justificación del Trabajo

En el mercado existen múltiples alternativas para la gestión de personal y sus tareas. Muchas de ellas incluyen funciones de fichaje, publicación de anuncios y documentos como las nóminas, horarios o gestión de ausencias y vacaciones, entre otras muchas.

En las últimas décadas, la **digitalización** ha transformado los sistemas de gestión de recursos humanos, especialmente en el ámbito de **control de fichaje** y **planificación** de turnos. Desde la entrada en vigor de la normativa española que obliga el registro de la jornada laboral (Real Decreto-ley 8/2919), las empresas han impulsado el uso de herramientas digitales que garanticen el cumplimiento de la ley y la **regularización del fichaje** de la jornada de los trabajadores en su día a día.

Este aspecto aumenta de importancia en trabajos como la hostelería o cualquier otra organización donde operen durante 7 días a la semana (aerolíneas, hoteles, hospitales, etc.). En estas áreas la gestión de los turnos de los empleados se convierte en un complejo problema que resolver, ya que en la organización del personal hay que incluir otros factores como los turnos rotativos, días de descanso (no más de 11 días seguidos trabajados según convenio, horas mínimas de descanso entre turno, etc.) y carga de trabajo que fluctúa a lo largo del día y que requiere una programación especial de la jornada laboral.

Varios estudios señalan que la planificación manual de turnos continúa siendo una ardua tarea, costosa y propensa a errores dentro de la gestión de personal. Esto implica que los responsables dedican entre un 20 y un 40% de su tiempo semanal a la elaboración de cuadrantes (Lakatos et al., 2025). A ello se

suma que deben tener en cuenta la disponibilidad real de los empleados, las necesidades de su negocio y restricciones legales, entre otros aspectos, que pueden generar ineficiencias y sobrecarga de trabajo, afectando a la productividad y la eficiencia del negocio.

En este contexto, las plataformas actuales de gestión de personal cubren dos grandes áreas: **control horario** (fichajes, geolocalización, etc.) y **gestión básica de turnos** (horarios semanales, solicitudes de ausencias, días de vacaciones, etc), como Factorial, BizneoHR o SesameHR. Sin embargo, no dan soluciones a la creación automática de horarios y no van más allá de la aplicación de plantillas o las validaciones de disponibilidad. Además, su coste y complejidad las posicionan **fuera del alcance** de la mayoría de pequeñas empresas (Ying et al., 2025).

Esta aplicación nace como respuesta a esta **necesidad en el mercado**: una aplicación web gratuita, de código abierto orientada a PYMEs, que ofrece las funcionalidades esenciales para la gestión de horarios con una interfaz clara e intuitiva. Se trata, por tanto, del desarrollo de un software de gestión de **Recursos Humanos (RRHH)**, **control horario** y planificación de personal (**Workforce Management**), orientado a pequeñas y medianas empresas (PYMEs) y microempresas en **España**. Desde el punto de vista académico, el proyecto representa una oportunidad óptima para integrar y demostrar las competencias adquiridas a lo largo del Máster en Desarrollo de Aplicaciones Web: diseño de arquitecturas software, desarrollo frontend y backend con tecnologías modernas, gestión de bases de datos, seguridad, accesibilidad y despliegue en entornos de producción.

1.2. Objetivos del Trabajo

En líneas generales, el objetivo de este trabajo de fin de máster (TFM) es el de diseñar, desarrollar y desplegar una aplicación web funcional que permita a pequeñas y medianas empresas gestionar turnos de trabajo y fichajes de forma eficiente, ofreciendo una herramienta accesible e intuitiva.

Específicamente, el desarrollo de este TFM se enfoca en cubrir los siguientes objetivos:

1. Evaluar las aplicaciones y herramientas actuales del mercado para la gestión de horarios y personal en pequeñas y medianas empresas, determinar las soluciones existentes y detectar las necesidades reales.
2. Diseñar el modelo de la aplicación y los componentes del sistema, siguiendo los principios de modularidad, escalabilidad y mantenibilidad.
3. Desarrollar un panel de administrador para la gestión de turnos, empleados y modificación manual de horarios (semanales/mensuales).
4. Desarrollar un portal de empleado donde pueda comprobar sus turnos, solicitar cambios, recibir notificaciones y fichar su entrada/salida.
5. Implementar un sistema de autenticación y autorización con diferenciación de los roles anteriormente descritos.
6. Garantizar la accesibilidad web del proyecto, cumpliendo con las pautas WCAG 2.1 en su nivel AA como mínimo.
7. Desplegar la aplicación web en un servidor público accesible mediante URL.

1.3. Impacto en sostenibilidad, ético-social y de diversidad

Por lo que respecta a la **Competencia de Compromiso Ético y Global (CCEG)**, el presente proyecto contribuye positivamente a los **ODS** (Objetivos de Desarrollo Sostenible, de la Agenda 2030 de la ONU) con los que la UOC está públicamente comprometida.

ODS 8 Trabajo decente y crecimiento económico	ODS 10 Reducción de las desigualdades	ODS 5 Igualdad de género	ODS 16 Paz, justicia e instituciones sólidas
ODS 9 Industria, innovación e infraestructura	ODS 12 Producción y consumo responsables	ODS 13 Acción por el clima	ODS 3 Salud y bienestar

Ilustración 1. Legenda de los ODS (Objetivos de Desarrollo Sostenible) en los que este TFM puede contribuir positivamente.

1.3.1. Sostenibilidad

Nuestra aplicación contribuye a la **sostenibilidad** en varias dimensiones. Si bien la naturaleza del proyecto es estrictamente digital, posee un impacto indirecto en el medioambiente, el consumo de recursos y la huella ecológica. Una planificación eficiente del personal y de su tiempo permite mejorar la eficacia de la plantilla y los **desplazamientos innecesarios** (ODS 13, acción por el clima), ya que una buena organización permite a los empleados anticipar sus viajes y traslados, reduciendo las emisiones de CO2 asociadas.

Por otra parte, la **digitalización** del fichaje horario y la publicación de documentos como las nóminas o avisos importantes reduce significativamente el uso del papel y la correspondencia impresa (ODS 12, producción y consumo responsables), así como el uso de tóner u otro material de oficina asociado.

1.3.2. Comportamiento ético y responsabilidad social

Esta aplicación también se puede justificar desde una perspectiva **ética**, al dar solución a la incertidumbre y la desorganización de aquellos trabajadores en

sectores con turnos partidos y horarios rotativos. Una mala gestión y comunicación de la planificación laboral de cada empleado reduce la incertidumbre y el **estrés asociado**, contribuyendo a una mejor calidad de vida y a unas **condiciones de trabajo más dignas** (ODS8, trabajo decente y crecimiento económico).

Además, introducir un **registro formal** y transparente, donde las organizaciones cumplan con las condiciones legales, como la detección automática de conflictos al incumplir con el descanso mínimo entre jornadas, contribuye directamente a la **transparencia y reducción de la arbitrariedad** (ODS, paz, justicia e instituciones sólidas), así como al cumplimiento de la normativa laboral, en particular, el Real Decreto Ley 8/2019.

Por último, la posibilidad de solicitar ausencias o cambios de turno y recibir notificaciones permite al empleado organizarse con antelación, mejorando su calidad de vida de manera directa y su relación con la empresa, permitiendo mejorar la productividad y **reducir la fatiga** (ODS 3, salud y bienestar).

En resumen, la aplicación tiene un impacto positivo claro en las condiciones laborales de los trabajadores, contribuyendo al cumplimiento normativo, el bienestar personal y al trabajo decente.

1.3.3. Diversidad, género y derechos humanos

En primera instancia, no parece haber una relación directa entre el proyecto y la perspectiva de género. Sin embargo, en trabajos como la hostelería u otros sectores como sanidad y servicios, para los que va enfocado esta aplicación, hay una presencia femenina clara y representada. Sobre ellas recae significativamente el peso de los cuidados al cargar de forma desproporcionada con estas responsabilidades.

Nuestra aplicación aborda la dimensión de género facilitando la **conciliación familiar y previsibilidad horaria** (ODS5, igualdad de género), ya que la anticipación y la opción de solicitar cambios de turno y ausencias tienen un impacto directo. Además, la aplicación **no asumirá roles de género** ni en la

iconografía, etiquetas ni uso de lenguaje. Los cargos y puestos de trabajo se denominarán de forma neutra o con denominación doble cuando corresponda.

La aplicación asumirá con el cumplimiento de las directrices **WCAG 2.1 AA** como requisito obligatorio, asegurando la **accesibilidad** de la web. Para ello tomará medidas como el uso de etiquetas semánticas en HTML, contraste de color correcto y textos alternativos descriptivos para los elementos no textuales, entre otros.

Es importante diseñar una aplicación que pueda ser usable para personas con distintas capacidades y perfiles tecnológicos (ODS10, reducción de desigualdades), ya que en entornos laborales se cuenta con una alta diversidad de plantilla (edad, nivel de estudios, idiomas, conocimientos tecnológicos, etc.). Por tanto, tomar medidas como las descritas anteriormente reduce la discriminación y asienta las bases de una aplicación inclusiva.

Además, la reducción de desigualdades (OD10) se ve favorecida por el carácter *open source* de la aplicación y la ausencia de coste de licencia, lo que favorecerá a las organizaciones sin ánimo de lucro, cooperativas y pequeñas empresas o entidades que no puedan asumir el coste de otras aplicaciones similares en el sector, reduciendo la brecha digital.

Por último, siguiendo las recomendaciones prescritas en la guía CCEG de la UOC, esta memoria emplea un lenguaje inclusivo de forma deliberada, utilizando formas neutras y/o dobles. La arquitectura y el diseño de la aplicación tampoco asumirá el género del usuario en ningún punto del sistema.

1.4 Enfoque y método seguido

El proyecto se ha iniciado con un breve estudio de mercado (**benchmarking**) sobre las herramientas actuales de gestión de turnos y recursos humanos. Este análisis nos permite conocer las funciones estándar, las carencias y las oportunidades de mercado y diferenciación. Esto, junto con la comunicación con el tutor, ha permitido definir un **MVP** realista y delimitado:

- Gestión de usuarios con roles diferenciados.
- Panel de administración para empleados, puestos y turnos.
- Creación y asignación manual de turnos.
- Portal del empleado con visualización de horarios.
- Diseño responsive y accesible (WCAG 2.1 AA).
- Despliegue en un servidor público.

De esta forma, quedan delegadas a funciones secundarias:

- Notificaciones a empleados
- Solicitud de cambios de turnos
- Gestión de incidencias

Se ha descartado la generación automática de turnos por tratarse de un problema algorítmico complejo que puede rebasar las competencias de este TFM, pero que se contempla como una **línea futura de ampliación**.

Por lo que respecta al apartado técnico, el proyecto adopta una arquitectura web moderna basada en:

- **Front-end en Angular**, orientado a usabilidad, accesibilidad y experiencia de usuario.

- **Back-end en Node.js**, con API propia para autenticación, gestión de datos y lógica de negocio.
- **Base de datos relacional** con PostgreSQL, adecuada para un dominio fuertemente estructurado (empleados, turnos, roles, horarios).
- **Diseño responsive** para garantizar uso en dispositivos móviles.
- **Buenas prácticas de seguridad**: JWT, cifrado de contraseñas, validación de entradas, HTTPS.

Se priorizará la creación de una aplicación **modular**, que permita la escalabilidad y facilite futuras ampliaciones y mejoras. El diseño de la interfaz se realiza bajo los principios UX/UI, a fin de crear una app intuitiva y clara de usar. Se crearán al menos dos perfiles principales: **rol de administrador** y **rol empleado**, con flujos y funciones específicos para cada uno.

En resumen, la elección de los apartados y características de esta aplicación, así como de las herramientas empleadas para su desarrollo, se debe a criterios de viabilidad y alineación con el tiempo disponible, a fin de garantizar una entrega final funcional, mantenible y coherente con los objetivos de este máster.

1.5. Planificación del Trabajo

Se definen los siguientes hitos que alcanzar:

Tabla 1. Orden de hitos y planificación del trabajo.

PEC / Hito	Descripción	Fecha límite	Entregables
PEC 1 - Análisis y planificación	Contextualización, objetivos, impacto, planificación, listado de funcionalidades	1 Marzo 2026	Memoria PEC1, doc. funcionalidades, checklist
PEC 2 - Diseño y arquitectura	Diseño de la arquitectura del sistema, wireframes, modelo de datos, API design	29 Marzo 2026	Memoria PEC2, diagramas, prototipo UI
PEC 3 - Implementación I	Backend: API REST con Node.js, autenticación, módulo de empleados y horarios básicos	26 Abril 2026	Memoria PEC3, código fuente, demo backend
PEC 4 - Implementación II, pruebas y despliegue.	Frontend Angular, creación de turnos, panel de empleados, notificaciones. Testing, correcciones, despliegue en servidor público, accesibilidad web (WCAG 2.1)	31 Mayo 2026	Memoria final, código fuente, aplicación desplegada, URL pública.
Defensa	Presentación oral del proyecto ante el tribunal evaluador	19 Junio 2026	Presentación, demo en vivo

Se comienza con una definición formal del trabajo y las pautas a seguir, evaluando su alcance, los objetivos y las herramientas que se emplearán estableciendo, en definitiva, la estructura de nuestro proyecto. En una segunda entrega se arranca el trabajo conforme a la planificación, documentando el proceso. En él incluiremos la viabilidad de la aplicación y el análisis de mercado. A mediados de abril se pretende contar ya con una versión beta del producto, incluyendo la grabación de un vídeo donde demostremos su funcionamiento. La entrega final incluirá tres elementos: el producto, la memoria y una presentación.

Descripción del hito	Inicio	Inicio real	Feb 2026			Mar 2026			Abr 2026				May 2026			Hores est.	% Real compl.		
			16/2	23/2	2/3	9/3	16/3	23/3	30/3	6/4	13/4	20/4	27/4	4/5	11/5	18/5	25/5		
PEC1: Planificación de tareas e investigación																		27	100%
1.1. Estudio, análisis y evaluación objetivos	18/02/2026	18/02/2026																8	100%
1.2. Análisis herramientas disponibles y MVP	01/03/2026	01/03/2026																6	100%
1.3. Elaboración de documentación	25/02/2026	01/03/2026																12	100%
1.4. ♦ Entrega PEC 1	01/03/2026	07/03/2026			♦													1	100%
PEC2: Investigación, análisis y resultados																		91	100%
2.1. Benchmarking, análisis de mercado	08/03/2026	08/03/2026																6	100%
2.2. Estudio arquitectura de la aplicación	08/03/2026	08/03/2026																8	100%
2.3. Análisis de viabilidad y casos de uso	10/03/2026	10/03/2026																8	100%
2.4. Desarrollo base de datos con PostgreSQL	14/03/2026	14/03/2026																15	100%
2.5. Desarrollo backend con Node.js	17/03/2026	17/03/2026																20	100%
2.6. Desarrollo frontend con Angular	26/03/2026	29/03/2026																15	100%
2.7. Elaboración wireframes y estudio UI	14/03/2026	14/03/2026																10	100%
2.8. Elaboración entregables PEC 2	23/03/2026	23/03/2026																8	100%
2.9. ♦ Entrega PEC 2	29/03/2026	26/03/2026						♦										1	100%
PEC3: Investigación, análisis y resultados 2																		92	72%
3.1. Desarrollo frontend con Angular	26/03/2026	29/03/2026																50	100%
3.2. Análisis de resultados y corrección errores	08/04/2026	08/04/2026																15	100%
3.4. Elaboración de la memoria final	14/04/2026	14/04/2026																10	100%
3.5. Despliegue en servidor público	23/04/2026	23/04/2026																10	100%
3.6. Tests unitarios	18/04/2026	23/04/2026																10	50%
3.7. Elaboración vídeo MVP	23/04/2026	23/04/2026																1	100%
3.5. Elaboración entregables PEC 3	23/04/2026	23/04/2026																10	80%
3.6. ♦ Entrega PEC 3	26/04/2026	26/04/2026										♦						6	50%
PEC4: Memoria final																		48	13%
4.1. Repaso consecución objetivos y resultados	27/04/2026																	6	0%
4.2. Conclusiones Trabajo Fin de Máster	01/05/2026																	8	0%
4.3. Elaboración de la memoria final	05/05/2026																	20	80%
4.4. Elaboración presentación vídeo	18/05/2026																	5	0%
4.5. Preparación defensa Trabajo Fin de Máster	22/05/2026																	8	0%
4.6. ♦ Entrega PEC 4	31/05/2026																♦	1	0%

Ilustración 2. Diagrama de Gantt para la planificación del proyecto. Elaboración propia.

Descripción del hito	Inicio	Inicio real	Riesgos identificados	Plan de contingencia
PEC1: Planificación de tareas e investigación				
1.1. Estudio, análisis y evaluación objetivos	18/02/2026	18/02/2026		
1.2. Análisis herramientas disponibles y MVP	01/03/2026	01/03/2026		
1.3. Elaboración de documentación	25/02/2026	01/03/2026		
1.4. ♦ Entrega PEC 1	01/03/2026	07/03/2026		
PEC2: Investigación, análisis y resultados				
2.1. Benchmarking, análisis de mercado	08/03/2026	08/03/2026		
2.2. Estudio arquitectura de la aplicación	08/03/2026	08/03/2026		
2.3. Análisis de viabilidad y casos de uso	10/03/2026	10/03/2026		
2.4. Desarrollo base de datos con PostgreSQL	14/03/2026	14/03/2026	Fallos de conexión o pérdida de datos	Implementar backups
2.5. Desarrollo backend con Node.js	17/03/2026	17/03/2026	Errores en autenticación o seguridad JWT	Añadir tests unitarios y validación de tokens en middleware
2.6. Desarrollo frontend con Angular	26/03/2026	29/03/2026	Desajuste visual o errores de compilación	Uso de Angular CLI y revisión de estilos globales
2.7. Elaboración wireframes y estudio UI	14/03/2026	14/03/2026		
2.8. Elaboración entregables PEC 2	23/03/2026	23/03/2026		
2.9. ♦ Entrega PEC 2	29/03/2026	26/03/2026		
PEC3: Investigación, análisis y resultados 2				
3.1. Desarrollo frontend con Angular	26/03/2026	29/03/2026	Desajuste visual o errores de compilación	Uso de Angular CLI y revisión de estilos globales
3.2. Análisis de resultados y corrección errores	08/04/2026	08/04/2026		
3.4. Elaboración de la memoria final	14/04/2026	14/04/2026		
3.5. Despliegue en servidor público	23/04/2026	23/04/2026	Bugs críticos en producción	Activar logs y rollback en Render
3.6. Tests unitarios	18/04/2026	23/04/2026		
3.7. Elaboración video MVP	23/04/2026	23/04/2026		
3.5. Elaboración entregables PEC 3	23/04/2026	23/04/2026		
3.6. ♦ Entrega PEC 3	26/04/2026	26/04/2026		
PEC4: Memoria final				
4.1. Repaso consecución objetivos y resultados	27/04/2026			
4.2. Conclusiones Trabajo Fin de Máster	01/05/2026			
4.3. Elaboración de la memoria final	05/05/2026		Retrasos por carga de trabajo	Planificar sesiones diarias de redacción y revisión con tutor
4.4. Elaboración presentación video	18/05/2026			
4.5. Preparación defensa Trabajo Fin de Máster	22/05/2026			
4.6. ♦ Entrega PEC 4	31/05/2026			

Ilustración 3. Planificación del proyecto con predicción de riesgos y plan de contingencia.

2. Material y métodos

En este apartado vamos a definir los recursos y metodologías necesarios para el desarrollo del proyecto. Esta metodología integra los procesos analíticos habituales de investigación del mercado y necesidades del usuario con la aplicación de las tecnologías actuales en el desarrollo de sitios y aplicaciones web. El objetivo es ofrecer una visión global de todo el proceso seguido, desde la definición de las funcionalidades principales hasta el desarrollo de un prototipo funcional.

2.1. Definición de diagramas y casos de uso

En primera instancia, vamos a realizar la definición y el modelado del sistema mediante **diagramas de casos de uso**. Estos nos permiten definir las bases funcionales de nuestra aplicación y los actores que interactuarán con ella.

Los casos de uso de la aplicación definen qué interacciones pueden darse entre los **actores** (los usuarios) y las funcionalidades ofrecidas por el **sistema**. Para ello, se ha seguido los **diagramas de casos de uso con notación UML** (del inglés, *Unified Modeling Language*) (*Guía Completa de Diagramas de Casos de Uso UML*, 2025).

Se han identificado dos actores principales en el sistema:

Tabla 2. Actores principales de casos de uso.

Actor	Descripción	Nivel de acceso
Empleado	Trabajador dado de alta en la empresa. Accede al portal de empleado para fichar y consultar sus turnos.	Lectura y escritura sobre sus propios datos únicamente.
Administrador	Responsable de la gestión de turnos y empleados. Tiene acceso a todos los módulos funcionales de la plataforma.	Acceso total, excepto la configuración técnica del sistema.

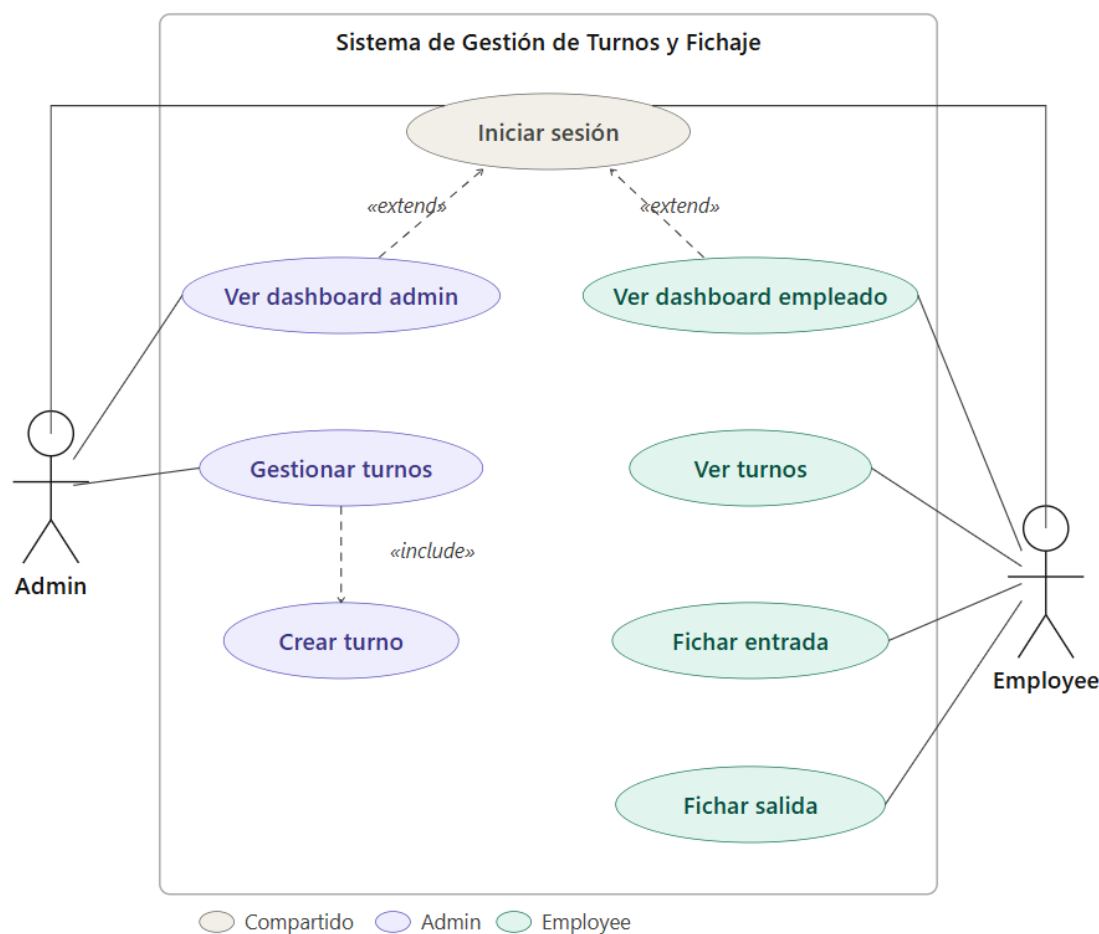


Ilustración 4. Diagrama de casos de uso siguiendo la nomenclatura UML. Elaboración propia.

Podemos ver en el diagrama cómo se dan las relaciones de casos de uso. Siguiendo la notación UML, los **óvalos** representan los casos de uso, es decir, las funcionalidades específicas de nuestro sistema. Mediante **líneas continuas** se marcan las interacciones entre el actor y el caso de uso. La etiqueta **<<include>>** representa la situación en la que un caso de uso incluye la funcionalidad de otro caso de uso (crear turno se incluye dentro de gestionar turnos). Del mismo modo, **<<extend>>** representa un comportamiento adicional de un caso de uso: el caso de uso Iniciar sesión, se amplía a los casos de uso Visualizar dashboard.

Estos casos de uso se han recogido en la Tabla 3, donde se detallan las especificaciones de los casos de uso correspondientes al núcleo funcional del sistema (MVP).

En resumen, los casos de uso ayudan a identificar y documentar los requisitos del sistema, asegurando que se cubren las funcionalidades necesarias, y funcionando como punto de partida para el estudio de la arquitectura de nuestra aplicación.

Tabla 3. Descripción detallada de casos de uso prioritarios.

ID del caso de uso	UC-1	UC-2	UC-3	UC-4	UC-5
Nombre del caso	Iniciar sesión	Fichar entrada/salida	Ver los turnos	Gestionar turnos	Visualizar dashboard
Actores	Empleado/Administrador	Empleado	Empleado	Administrador	Empleado/Administrador
Precondiciones	El usuario debe estar registrado en el sistema. Las credenciales deben ser correctas.	El empleado debe estar autenticado	El empleado debe estar autenticado	El administrador debe estar autenticado.	Usuario autenticado
Flujo principal	1. El usuario introduce email y contraseña 2. El sistema procesa la acción y se abre el dashboard correspondiente (admin/dashboard o employee/dashboard).	En la pantalla de fichaje, selecciona "Entrada" o "Salida". El sistema registra la marca temporal.	Accede a "Mis turnos". Desde el backend, el sistema obtiene los turnos y los muestra en lista.	Accede a "Gestión de turnos". Selecciona "crear turno". Introduce el nuevo turno (día, hora, empleado). El sistema valida los datos, revisando validaciones. Almacena el nuevo turno.	Tras autenticarse visualiza su panel de control respectivo a su rol.
Postcondiciones	El token JWT queda almacenado y el usuario queda autenticado y con los permisos correspondientes.	El fichaje queda registrado en la base de datos.	El empleado visualiza sus turnos.	El turno queda registrado.	El usuario accede al panel de control.
Flujos alternativos	Si las credenciales son incorrectas el servidor rechaza la autenticación, mostrando un mensaje de error; se permanece en la pantalla de login.	Si el empleado trata de fichar dos entradas consecutivas, se muestra un mensaje de error.	Si no hubiera turnos asignados, se muestra un mensaje.	Si se dan conflictos de turno (solapamiento o incumplimiento de las condiciones de convenio) se muestra un mensaje de error. Si se intenta guardar un turno con datos incompletos, se muestra un mensaje de error.	Token expirado: se redirige a la pantalla de login.

2.2. Estudio de usabilidad

La norma **ISO 9241-11** define **usabilidad** como “la capacidad que tiene un producto software para ser entendido, aprendido, operable, atractivo para el usuario y conforme a estándares/guías, cuando es utilizado bajo unas condiciones específicas” (ISO 9241-11, 2018). Esta definición sitúa la usabilidad como un atributo fundamental del software, que debe ser: fácil de aprender, entender y operar, atractivo y conforme a estándares y guías (Olga Carreras, 2012). Estas dimensiones han guiado tanto el diseño de la interfaz como las métricas de su evaluación.

El análisis de la usabilidad consistirá en evaluar cómo de intuitivo y eficiente es el diseño de la aplicación en base a la interacción con usuarios. Es decir, analizaremos la **experiencia de usuario**. Esto es un factor crítico dado que los usuarios de la aplicación, empleados reales, no tienen por qué tener conocimientos técnicos previos y, por tanto, debe diseñarse una interfaz clara, intuitiva y accesible.

Estos análisis de accesibilidad pueden evaluarse siguiendo diferentes estrategias y directivas. Por una parte, se suele llevar a cabo un análisis sin usuarios reales mediante la **evaluación de los 10 principios heurísticos de Nielsen**. Este análisis permite la detección de problemas en las primeras fases del desarrollo y, dado que no se necesita un grupo experimental, es rápido de realizar, económico y compatible con otras técnicas de evaluación (Carlos Vicente, 2024).

Por otra parte, se realizará una prueba de usabilidad **con usuarios reales**. Estas personas realizarán tareas que coinciden con los casos de uso de nuestro MVP definidos en el apartado anterior.

2.2.1. *Evaluación heurística*

La **evaluación heurística** consiste en el estudio y la evaluación de la interfaz por uno o varios expertos en usabilidad, en base a una serie de reglas y principios de diseño preestablecidos (Carlos Vicente, 2024). Aunque puede realizarse en distintos estadios del desarrollo de la aplicación, vamos a realizarlo durante esta

primera etapa de diseño sobre el prototipo de alta fidelidad (Figma) antes del inicio del desarrollo de código.

Estos principios sometidos a análisis, que se denominan heurísticos o de usabilidad, fueron definidos por Jakob Nielsen y han funcionado como referencia internacional en UX (Jakob Nielsen, 1994). Estas reglas serían:

Tabla 4. Los 10 heurísticos de Nielsen y su aplicación en PlanifyApp.

#	Heurístico	Aplicación en Planify
H1	Visibilidad del estado del sistema	El sistema muestra en todo momento el estado del fichaje (dentro/fuera de turno), indicadores de carga durante las peticiones al servidor y mensajes de confirmación tras cada acción.
H2	Correspondencia con el mundo real	Se utiliza terminología propia del sector (turno, fichar, cuadrante) en lugar de tecnicismos. Los iconos representan conceptos familiares: reloj para el fichaje, calendario para los turnos.
H3	Control y libertad del usuario	Todas las acciones destructivas (eliminación de turno, modificación de fichaje) requieren confirmación previa. Se provee navegación hacia atrás en todos los flujos.
H4	Consistencia y estándares	Se sigue el sistema de diseño de Material Design como referencia. Los patrones de navegación, tipografía y color son consistentes en toda la aplicación.
H5	Prevención de errores	Los formularios incluyen validación en tiempo real (ej: solapamiento de turnos, descanso mínimo insuficiente) antes de permitir el envío. Los campos obligatorios están claramente marcados.
H6	Reconocer antes que recordar	Los turnos del día se muestran en el dashboard al iniciar sesión, sin necesidad de navegar. Los formularios de creación de turno incluyen sugerencias de horarios habituales.
H7	Flexibilidad y eficiencia de uso	Para el MVP, la eficiencia se prioriza mediante una arquitectura de navegación plana (máximo 2-3 clics para cualquier acción principal). Las funciones avanzadas (filtros, exportación) se reservan para versiones futuras.
H8	Estética y diseño minimalista	La interfaz elimina elementos decorativos innecesarios. El diseño se centra en las acciones principales de cada rol, evitando la sobrecarga de información.
H9	Ayuda a reconocer y recuperarse de errores	Los mensajes de error son descriptivos y orientados a la solución ('El descanso mínimo entre jornadas es de 12 horas. La hora de entrada introducida no cumple este requisito'). Se evitan códigos de error técnicos.
H10	Ayuda y documentación	Se incluye una sección de ayuda contextual en cada módulo.

Los resultados de la evaluación heurística aplicada al prototipo se recogen en el apartado 3. Resultados.

2.2.2. Prueba de usabilidad con usuarios reales

Las **pruebas de usabilidad con usuarios reales** permiten observar el comportamiento real de las personas representativas del público objetivo cuando interactúan con la aplicación. Estos usuarios deben ejecutar tareas basadas en los casos de uso que definen el MVP, y serán guiados por un moderador que observa y registra las dificultades encontradas.

Igual que con la evaluación heurística, la prueba se realizará sobre el prototipo de alta fidelidad en Figma. Se ha reclutado a 5 participantes, ya que, según Nielsen, con 5 usuarios es posible cubrir el 85% de los problemas de usabilidad (Paulo Cesar Vicente Da Luz, 2023). Aunque se trata de una guía y no de una regla inquebrantable, este grupo reducido se ha considerado suficiente dada la naturaleza de la prueba y la poca complejidad de las tareas a realizar.

Las tareas realizadas, alineadas con los casos de uso del MVP, son:

Tabla 5. Tareas que realizar en la ejecución de la prueba.

ID	Perfil	Tarea	Criterio de éxito
T-01	Ambos	Inicia sesión con tus credenciales.	Acceso al dashboard en < 60 seg. sin ayuda.
T-02	Empleado	Consulta los turnos que tienes asignados esta semana.	Localiza el horario semanal en < 90 seg.
T-03	Empleado	Ficha tu entrada como si fuese el inicio de tu jornada.	Completa el fichaje en < 30 seg. sin errores.
T-04	Administrador	Crea un turno de mañana (8:00-16:00) para el empleado María López el próximo lunes.	Turno creado correctamente en < 3 min.
T-05	Administrador	Modifica el turno del empleado Juan García del jueves, cambiando la hora de fin a las 18:00.	Modificación completada sin errores.

Para evaluar el éxito de la prueba se ha determinado qué porcentaje de participantes completan las tareas asignadas y en cuánto tiempo. Durante las sesiones, se ha tomado nota de las dificultades encontradas, los errores cometidos y los comentarios esporádicos de los participantes que puedan ser informativos.

Los resultados obtenidos se incluyen en el apartado 3. Resultados.

2.2.3. Principios de accesibilidad (WCAG 2.2 nivel AA)

Las **Pautas de Accesibilidad para el Contenido Web (WCAG)**, del inglés *Web Content Accessibility Guidelines*) se desarrollan a través del proceso **W3C** en cooperación con individuos y organizaciones con el fin de proporcionar un estándar para la accesibilidad del contenido web. Con ellas, se garantiza la accesibilidad de los contenidos web a todos los usuarios, diseñando, por tanto, una web **inclusiva** (Alejandro Moreno Montero, 2025).

Desde su origen en 1999 se han creado múltiples versiones. Para este trabajo tomaremos la versión WCAG 2.1 nivel AA, que es la exigida actualmente por el Real Decreto 1112/2018 y establece criterios como el contraste adecuado, iconografía clara, navegación sencilla y compatibilidad con diferentes dispositivos y tamaño de pantalla (*Nueva Normativa de Accesibilidad Web: ¿qué Exige La Nueva Ley y Cómo Te Afecta?*, 2025). Adaptarse a estas pautas no solo evita sanciones legales: crear una web accesible amplía el mercado, mejora el SEO, refuerza la reputación de la marca y reduce costes a largo plazo al consolidar un código limpio y pautado. Algunas de estas pautas que se han aplicado para el desarrollo de PlanifyApp han sido:

- **Contraste de color.**
- **Etiquetas en formularios**, mediante atributos 'for' o 'aria-label'.
- **Mensajes de estado**, como mensajes de error y confirmación mediante atributos ARIA.
- **Textos alternativos** para todos los elementos no textuales (imágenes, iconos, etc.).

Para su evaluación se realizará una validación de la web mediante la herramienta online WAVE (WebAIM), que proporciona un análisis automático y un informe detallado sobre errores de accesibilidad, advertencias y problemas de contraste, entre otras indicaciones. Dado que esta herramienta no sustituye una auditoría completa, se completará con una revisión manual. Los resultados se recogen en el apartado 3. Resultados. En definitiva, combinar los métodos de

evaluación heurística con test de usabilidad en usuarios reales y las pautas de accesibilidad WCAG permite obtener una visión completa de la accesibilidad de nuestra interfaz y del comportamiento real de los usuarios con ella, y nos permite **detectar** y **corregir** errores de diseño con bajo coste **antes** de la etapa de desarrollo de código.

2.3. Wireframes, diseño inicial y diagramas de flujo

Para el diseño de la interfaz de PlanifyApp se ha seguido la metodología de **Diseño Centrado en el Usuario** (DCU, normativa ISO 9421-210), donde cada decisión de diseño se realiza en base a las necesidades y limitaciones del usuario. Podemos dividir el proceso de diseño en tres etapas: creación de los primeros diseños en papel, creación de los wireframes estáticos de baja calidad en escala de grises (Figma) y desarrollo del prototipo de alta fidelidad (Figma). A continuación, trataremos cómo ha sido el proceso de diseño y la justificación de las decisiones tomadas.

2.3.1. *Wireframes y prototipos en Figma*

Siguiendo un proceso estructurado de dos niveles de fidelidad, en primer lugar, se ha elaborado **5 wireframes** de baja fidelidad (en papel y, después, en Figma) para definir la jerarquía principal de la aplicación y la distribución de los elementos sin incluir aún decisiones estéticas. Esta etapa es vital para establecer un primer boceto del **flujo de navegación**, y nos ha permitido extraer el **sitemap** correspondiente.

A continuación, se ha diseñado 5 wireframes de alta fidelidad que incluyen decisiones estéticas y detalles visuales con todas las pantallas del MVP con interacciones navegables. Los flujos que cubren son:

- Autenticación. Pantalla de login y redirección según el rol.
- Dashboard del empleado: botón de fichaje y calendario.
- Empleado: visualización de turnos.

- Dashboard del administrador: visualización y gestión de turnos.
- Administrador: listado de empleados.

El prototipo interactivo en Figma está disponible en el siguiente [enlace](#). Los wireframes de baja y alta fidelidad se encuentran en el apartado 3. Resultados.

2.3.2. Diagramas de flujo de navegación

Los **diagramas de flujo de navegación** representan el camino que sigue un usuario, dentro de la aplicación, para realizar una tarea determinada (autenticarse, crear turno, etc.). En el diagrama creado (Ilustración 4) se muestran las tareas alineadas con las funciones del MVP (T-01 a T-05). Se muestran también las pantallas involucradas. Hay tres flujos principales:

- **Flujo de autenticación**, compartido.
- **Flujo de fichaje** (empleado): acceso al dashboard → botón “Entrar” → registro en la base de datos → actualización del estado en la pantalla.
- **Flujo de gestión de turnos** (administrador): “Gestión de turnos” → botón “Crear nuevo turno” → introducción de datos → validación del convenio (backend) → confirmación o mensaje de error por pantalla.

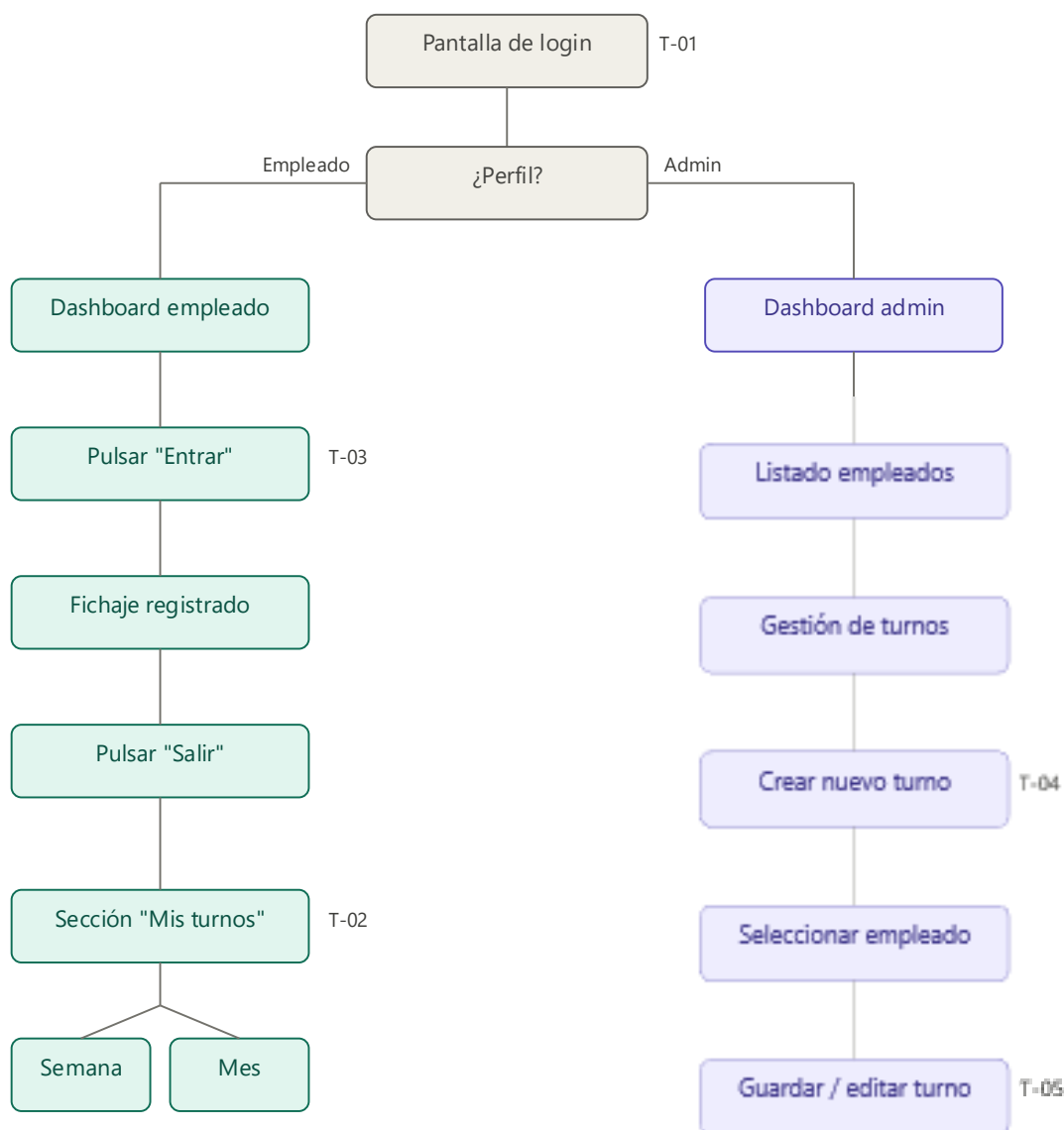


Ilustración 5. Diagrama de flujo. Elaboración propia.

2.4. Estudio de arquitectura de la aplicación

En este punto evaluaremos la **organización interna de la aplicación**: cuáles son las capas que la constituyen y cómo se comunican entre sí. Es importante definirla porque crear un producto bien estructurado implica que sea fácil de desarrollar y, especialmente, moduable. Esto es una característica muy importante en general y, en especial, en este TFM donde el límite de tiempo y recursos generará una aplicación con funcionalidades más básicas de lo

deseado. Además, nos permite identificar qué partes del sistema son críticas y cuáles podemos desarrollar después (Clements, 1999).

2.4.1. Patrón arquitectónico: cliente-servidor de tres capas

PlanifyApp adopta una arquitectura estándar en aplicaciones web modernas de **cliente-servidor de 3 capas** (*three-tier architecture*):

- **Frontend (capa de presentación):** gestiona la interfaz de usuario y la interacción con el navegador y se ha desarrollado con Angular. El frontend se comunica con el backend a través de peticiones HTTP/HTTPS con una API REST.
- **Backend (lógica de negocio):** será el motor de la aplicación. Se ha desarrollado con Node.js, Express y TypeScript. Recibe las peticiones del frontend, consulta o modifica la base de datos y se aplican las validaciones, el control de autenticación y las restricciones del convenio para la gestión de turnos.
- **Base de datos:** PostgreSQL almacena los datos (usuarios, fichajes, etc.), garantizando la integridad referencial con claves primarias y foráneas.

2.4.2. Modelo de datos y API REST

Se ha creado cuatro tablas principales que recogen el modelo de datos relacional de PlanifyApp. Sus relaciones se recogen en el diagrama Entidad-Relación (ER) creado con pgAdmin4.

Tabla 6. Entidades principales del modelo de datos.

Entidad	Atributos principales	Descripción
USUARIOS	id, nombre, email, password_hash, role, position, created_at	Almacena todos los usuarios del sistema (empleados y administradores). El campo role usa una restricción CHECK para garantizar que solo existen los valores 'ADMIN' o 'EMPLOYEE'. Las contraseñas se almacenan como hash (bcrypt), nunca en texto plano.
HORARIOS	id, employee_id (FK), start_time, end_time	Registra los turnos asignados a cada empleado. Una restricción CHECK garantiza que end_time > start_time. Las validaciones de convenio (descanso mínimo, solapamiento) se aplican desde el backend antes de insertar el registro.

Entidad	Atributos principales	Descripción
FICHAJES	id, employee_id (FK), schedule_id (FK), clock_in, clock_out, total_minutes	Registra las marcas temporales de entrada y salida de cada empleado. El campo clock_out es NULL hasta que el empleado ficha la salida. Se vincula al turno (schedule_id) para asegurar que el fichaje se realiza dentro del periodo laboral previsto.
FICHAJES_AUD	id, fichaje_id (FK), employee_id, clock_in, clock_out, changed_at, changed_by (FK), action	Tabla de auditoría inmutable que registra cualquier inserción, modificación o eliminación en la tabla FICHAJES. Permite garantizar la integridad laboral de los registros y disponer de trazabilidad completa en caso de litigio legal. El campo changed_by indica qué usuario realizó el cambio.

Para realizar la comunicación entre el frontend (Angular) y el backend (Node.js) se ha creado una **API REST** con las convenciones estándar de HTTP (métodos GET, POST, PUT, DELETE y los códigos de estado):

Tabla 7. Endpoints de la API REST para el MVP de PlanifyApp.

Método	Endpoint	Descripción	Acceso
POST	/api/auth/login	Autentica al usuario y devuelve un token JWT.	Público
GET	/api/users	Lista todos los usuarios/empleados.	Solo ADMIN
POST	/api/users	Crea un nuevo usuario.	Solo ADMIN
PUT	/api/users/:id	Actualiza los datos de un usuario.	Solo ADMIN
DELETE	/api/users/:id	Elimina un usuario del sistema.	Solo ADMIN
GET	/api/schedules	Lista todos los turnos (admin) o los propios (empleado).	ADMIN y EMPLOYEE
POST	/api/schedules	Crea un nuevo turno para un empleado.	Solo ADMIN
PUT	/api/schedules/:id	Modifica un turno existente.	Solo ADMIN
DELETE	/api/schedules/:id	Elimina un turno.	Solo ADMIN
POST	/api/timelog/clock-in	Registra la entrada del empleado.	Solo EMPLOYEE
POST	/api/timelog/clock-out	Registra la salida del empleado.	Solo EMPLOYEE
GET	/api/timelog	Consulta el historial de fichajes.	ADMIN y EMPLOYEE

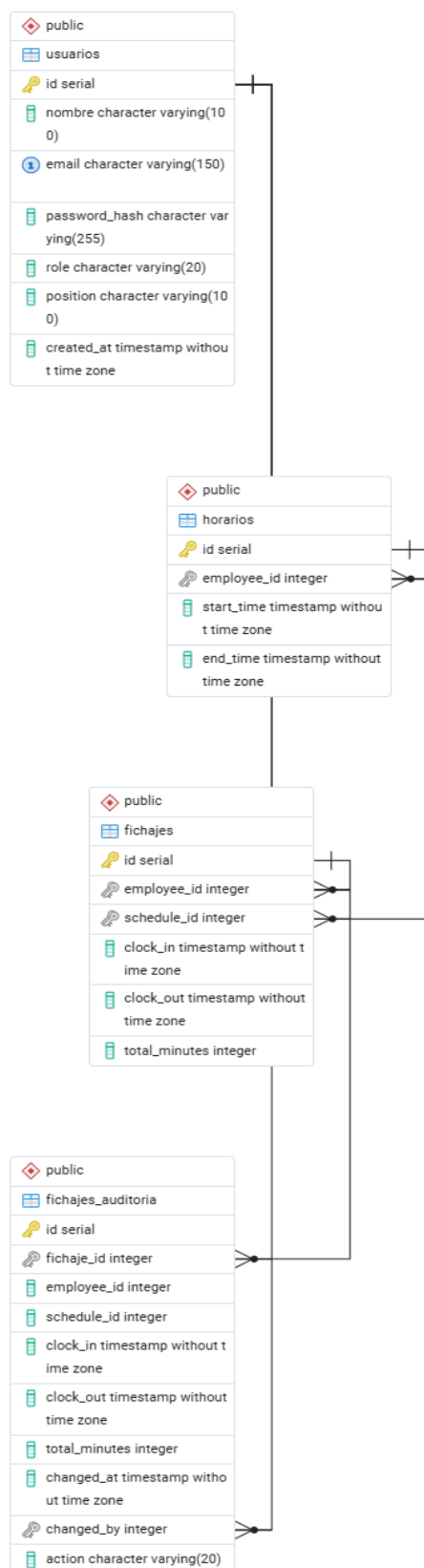


Ilustración 6. Diagrama Entidad-Relación de las principales entidades del modelo de datos de PlanifyApp.

2.5. Benchmarking, análisis de competidores y público objetivo

El **benchmarking** se define como el proceso sistemático de comparar los productos, servicios o prácticas empresariales de una organización con los de sus competidores. Su principal objetivo es identificar aquellas oportunidades de mejora y buenas prácticas que permitan alcanzar niveles superiores de eficiencia y eficacia (Adrià Turneu Puig, 2024).

En el caso de la creación de sitios y aplicaciones web no solo debemos centrarnos en la evaluación de la **funcionalidad** de las herramientas competidoras, sino también en aspectos como el **rendimiento**, la **usabilidad** y la experiencia de usuario.

A continuación, realizaremos un doble análisis: por un lado, caracterizaremos el mercado actual competitivo; por otro, evaluaremos la viabilidad de desarrollar una plataforma alternativa *open source* gratuita y adaptada al marco jurídico español.

2.5.1. Análisis de mercado

El 99,8% del tejido empresarial español son PYMEs, que generan el 62% del empleo y el 61% del PIB. Sin embargo, el 35% de estas empresas aún no han adoptado ninguna solución digital de RRHH (General de Industria PYME, 2030). Esto supone una situación de **riesgo**: desde 2019 el **registro horario** en España es obligatorio y, desde este año, dicho registro debe ser **digital**. Además, las empresas deben adoptar medidas para cumplir con la reducción de la jornada laboral a **37,5 horas** semanales (*Todo Sobre La Nueva Ley de Control Horario de Los Trabajadores*, 2025).

Si bien la digitalización de RRHH es una necesidad operativa para cualquier empresa independientemente de su tamaño, para las microempresas, que constituyen el 95,2% de las PYMEs (incorporar referencia datos INE) supone una **barrera económica significativa**. Actualmente, las soluciones HR para PYMES son de pago y tienen precios que oscilan entre 4€ y 12€ al mes por empleado. En términos relativos, esto supone una gran inversión y gasto para

las empresas pequeñas que no pueden contar con un departamento RRHH dedicado.

Este contexto brinda un hueco en el mercado para crear una solución *open source* gratuita, pensada específicamente para PYMEs locales, que se están digitalizando por primera vez, y adaptada al marco jurídico español cuyas nuevas reformas catapultan la adopción de estas soluciones.

2.5.2. Competidores directos

Para el análisis de competidores se ha tenido en cuenta los siguientes criterios de inclusión:

- Operativa en el mercado español.
- Funcionalidades solapantes en, al menos, un 60% de las proyectadas para nuestro producto: control horario, registro de fichaje, gestión de ausencias y vacaciones, planificación de turnos.
- Disponibilidad de información viable para el análisis.

Se ha evaluado a los competidores directos en base al control horario, la gestión de turnos y RRHH en general, analizando sus puntos fuertes y debilidades y su posición de competencia con nuestro proyecto. Las dimensiones que se han analizado se recogen en la siguiente tabla.

Tabla 8. Evaluación de competidores directos.

	Precio desde	Micro (<10)	PYME (10-50)	Nóminas ES	Registro jornada	Turnos
Factorial HR	4–5,50 €/empl.	Sí	Óptima	Sí	Sí	Sí
Sesame HR	2,50 €/empl.	Sí	Óptima	Sí	Sí	Sí
TramitApp	1–2 €/usuario	Ideal	Sí	No	Sí	Básico
Holded	14,50 €/empresa	Ideal	Básico	Básico	Básico	No
Personio	3,76 €/usuario	Posible	Óptima	Sí	Sí	Básico
Bizneo HR	9 €/empl.	Excesivo	Óptima	Sí	Sí	Sí
Kenjo	Cotización	No	Buena	Básico	Sí	Básico
Connecteam	Gratis/10 users	Gratis	Sí	No	Sí, GPS	Sí
Zoho People	~1,25 €/user	Sí	Sí	Básico	Sí	Básico

Dado que ninguna plataforma es *open source*, se ha omitido el dato de la tabla a fin de simplificarla.

Podemos ver que **Factorial HR** (Factorial, 2026) y **Sesame HR** (Sesame HR, 2026), muy extendidas en el mercado, son propuestas competidoras con perfiles muy similares, aunque la segunda posee menos funcionalidades.

Por otra parte, **TramitApp** (TramitApp, 2026) y **Holded** (Holded, 2026) son las opciones más económicas, diseñadas para micro y pequeñas empresas. Sin embargo, se centran en dar solución a la digitalización de documentos laborales y control de fichaje, pero no en la creación de turnos ni planificación de empleados.

Podemos concluir que el competidor directo más amenazante es **Factorial HR**, ya que nació enfocado a PYMEs (aunque escala a empresas de hasta 500 empleados) y es líder en el mercado actual. Cuenta con un control horario y fichaje con geolocalización, gestión de turnos, vacaciones y ausencias, a un precio competitivo pero adaptable a medianas empresas. Sin embargo, supone un **coste demasiado restringido** para microempresas y la aplicación cuenta con módulos de pago no incluidos en la tarifa. Además, se ha notificado problemas de usabilidad y fallos en la atención al cliente (Susan Snipes, 2026). Precisamente, nuestra alternativa *open source* gratuita puede competir con ella en este segmento de mercado. Sin embargo, cumple de forma muy completa con la normativa y el marco legal español (SEPE, Seguridad Social, registro de jornada, firma electrónica y jornada de 37,5).

Por otra parte, **TramitApp** destaca por ser la opción más económica del mercado. Sin embargo, más allá del fichaje móvil, cuenta con una lista de funciones muy limitadas. Lo mismo ocurre con **Holded**: ofrece soluciones brillantes para PYMEs y autónomos que necesitan herramientas de automatización e integración para la gestión de nóminas, impuestos y otros aspectos referentes a la contabilidad. Sin embargo, dejan de lado la gestión de turnos y la planificación de personal de una forma óptima y sencilla.

Otra gran competidora es **SesameHR**, ya que está enfocado en empresas pequeñas y cuentan con un UX simple y rapidez de implantación. Aunque gusta por su sencillez, muchos usuarios reportan la falta de soporte y la presencia de

errores en la plataforma que dificultan su implementación (Capterra, 2025) (GetApp, 2026).

De este análisis podemos extraer que sí existe una **brecha en el mercado**, ya que existe una demanda insatisfecha de soluciones que resuelvan, de forma simultánea, los siguientes tres puntos:

1. 100% gratuita y *open source*.
2. Adaptada al marco jurídico laboral español (RDL 8/2019).
3. Con una UX sencilla específica para PYMEs y microempresas que no puedan costear un departamento RRHH dedicado.

2.5.3. Análisis DAFO

Tras el análisis de mercado y las conclusiones obtenidas tras el benchmarking, se ha realizado un **análisis DAFO** para evaluar las fortalezas y debilidades de nuestra aplicación planteada, así como las oportunidades y amenazas del entorno competitivo.

Tabla 9. Análisis DAFO de Planify.

FORTALEZAS	DEBILIDADES
- Solución 100% gratuita y open source. - Adaptada al marco jurídico laboral español (RDL 8/2019, 37,5 h/semana). - UX sencilla, orientada a perfiles no técnicos. - Diseño accesible (WCAG 2.1 AA). - Código modular y escalable (Angular + Node.js). - Transparencia total: auditoría de fichajes integrada.	- Proyecto unipersonal: recursos de desarrollo limitados. - Sin generación automática de turnos en el MVP. - Sin soporte de nóminas ni integración con la Seguridad Social. - Ausencia de plan de soporte técnico formal. - Reconocimiento de marca nulo en el mercado.
OPORTUNIDADES	AMENAZAS
- Obligatoriedad del fichaje digital desde 2025. 35% de PYMEs todavía sin herramienta digital de RRHH. - Sectores con alta rotación (hostelería, sanidad, retail) sin solución asequible. - Comunidad open source: potencial de contribuciones externas. - Ampliación futura con algoritmos de generación automática de turnos.	- Plataformas SaaS consolidadas (Factorial, Sesame) con recursos para reducir precios. - Connecteam ofrece plan gratuito para hasta 10 usuarios. - Riesgo de obsolescencia si la normativa española cambia. - Desconfianza del mercado ante soluciones sin soporte comercial.

2.5.4. Público objetivo y User Persona

Con el fin de crear una web que se adapte a las necesidades de los usuarios y orientar las decisiones de diseño, se ha creado dos **user persona** representativos de los perfiles demandantes de la aplicación. Un user persona es una representación ficticia de un usuario típico o ideal de un producto, servicio o sitio web, y pueden perfilarse obteniendo información sobre sus motivaciones, sus frustraciones, sus deseos, etc. (User Persona, 2025).

Tabla 10. User persona 1: rol administrador.

Campo	Detalle
Nombre	Marta García, 38 años
Rol en la app	Administradora
Perfil profesional	Encargada de sala en un restaurante familiar con 12 empleados. Lleva 10 años en el sector. No tiene formación en gestión de software, aunque usa el móvil con fluidez.
Motivaciones	Quiere dejar de hacer los cuadrantes de turno a mano en Excel. Se olvida de avisar a los empleados cuando hay cambios y esto genera conflictos. Necesita algo que le ahorre tiempo y le dé visibilidad.
Frustraciones	Las herramientas que ha probado son caras o muy complejas. Cuando se va de vacaciones, la persona que la sustituye no sabe gestionar el sistema. No tiene tiempo para aprender herramientas nuevas.
Objetivo principal	Crear y asignar turnos en menos de 10 minutos. Que los empleados puedan ver sus horarios desde el móvil sin tener que llamarla.
Frase característica	"Si tuviese algo sencillo que no me costase un dineral, ya lo habría puesto hace años."

Tabla 11. User persona 2: rol empleado.

Campo	Detalle
Nombre	Adrián Moreno, 24 años
Rol en la app	Empleado
Perfil profesional	Camarero en un hotel de playa. Trabaja a turnos rotativos. Usa el móvil constantemente y es usuario habitual de apps como WhatsApp, Instagram y Google Maps.
Motivaciones	Quiere saber con antelación cuándo trabaja para poder organizarse. Le molesta enterarse de los cambios de turno por WhatsApp el día anterior.

Frustraciones	A veces ficha tarde porque no llega a tiempo a la máquina de la entrada. No sabe cómo solicitar cambios de turno de forma oficial. La empresa le envía el horario en PDF y es difícil de ver en el móvil.
Objetivo principal	Consultar su horario desde el móvil. Fichar la entrada y salida sin complicaciones.
Frase característica	"Solo quiero saber a qué hora entro mañana sin tener que preguntar."

2.6. Estudio de Viabilidad

El estudio de viabilidad de un proyecto consiste en realizar un análisis preliminar para determinar si es técnicamente posible, financieramente rentable y operativamente sostenible antes de invertir tiempo y recursos en su realización (*5 Pasos Para Realizar Un Estudio de Viabilidad de Un Proyecto*, 2024). En este capítulo vamos a analizar las tres áreas más importantes de un estudio de viabilidad: técnica, económica y del mercado.

2.6.1. Viabilidad técnica

En este apartado se evalúa si la empresa dispone de la tecnología y las herramientas necesarias para realizar el proyecto. Esto no solo incluye los materiales físicos, también se evalúa si se tiene los conocimientos necesarios para el desarrollo. Para ello, debemos retomar las funcionalidades de nuestro MVP y las accesorias:

- **Roles diferenciados** de empleado y administrador
- **Acceso web** desde el navegador, sin instalación local
- **Gestión de horarios** y asignación manual de turnos.
- **Almacenamiento de datos**: base de datos para empleados y horarios
- **Seguridad básica**: autenticación.
- **Fichaje**

Estos requisitos definen el alcance técnico mínimo para que la solución resulte viable y útil. Las tecnologías seleccionadas son **Node.js** con **Angular** para el backend y frontend, respectivamente. La elección se debe a que son herramientas que cuentan con gran soporte y una comunidad amplia. Además,

han sido utilizadas a lo largo del máster en múltiples ocasiones de forma integrada, lo que añade valor a la facilidad de uso y a su gran implementación en el área actual de desarrollo web. Para el frontend, además, se aplicarán los conocimientos obtenidos en JavaScript, HTML y CSS, que suponen el estándar actual. Por último, se usará **PostgreSQL** para la base de datos y Git y Github para el control de versiones. En definitiva, se usarán herramientas gratuitas, accesibles y conocidas que minimizan el riesgo técnico.

Uno de los **riesgos** a los que podemos enfrentarnos es la falta de **viabilidad temporal**. Por eso es importante definir bien el MVP y organizar y estimar correctamente el tiempo disponible y el requerido para el desarrollo del proyecto. Para mitigarlo, se ha definido un **alcance mínimo** que incluye la gestión básica de turnos y el control horario.

2.6.2. *Viabilidad económica*

Dado que se han elegido **softwares gratuitos** y de código abierto (base de datos, editor de código, repositorio, etc) y se desarrolla con el hardware propio ya disponible, el gasto para la realización de este TFM es **cero**.

2.6.3. *Viabilidad de mercado*

Como ya hemos visto, se ha analizado la competencia y se ha realizado un análisis de mercado a fin de determinar si hay o no una **demanda real**. Se ha concluido que sí: la aplicación proyectada puede responder a un segmento vacío del mercado y a las necesidades crecientes de sectores como la hostelería, donde se requiere:

- **Obligatoriedad de fichaje digital** (desde 2025) para todas las empresas, independientemente de su tamaño o sector.
- La llegada de la **jornada de 37,5 horas** semanales, que obliga a llevar un nuevo control de las horas trabajadas.
- El carácter **open source** elimina la barrera económica que suponen el coste de licencia de otras plataformas actuales del mercado.
- La orientación a sectores con alta rotación de personal y turnos complejos (hostelería, comercio, sanidad) concentra la demanda en segmentos con urgencia operativa

2.7. Desarrollo de la aplicación

En este apartado se describe desarrollo técnico de PlanifyApp, las decisiones de configuración y la implementación del backend y del frontend, así como de las herramientas utilizadas. Se ha tratado de describir siguiendo un orden cronológico.

2.7.1. Entorno de desarrollo

El desarrollo de esta aplicación responde a la arquitectura típica **cliente-servidor**, un modelo de diseño de sistemas distribuidos en el que las responsabilidades se reparten entre el **cliente**, que solicita los servicios, y el **servidor**, que los proporciona. La descripción del entorno de trabajo se recoge en la siguiente tabla:

Tabla 12. Herramientas utilizadas para el entorno de desarrollo de PlanifyApp

Categoría	Herramienta / Versión	Justificación
Sistema operativo	Windows 11	Entorno de desarrollo habitual.
Editor de código	Visual Studio Code 1.9x	Editor gratuito, extensible y con soporte nativo para TypeScript, Angular y Node.js.
Gestor de versiones	Git + GitHub	Control de versiones distribuido. El repositorio remoto permite el seguimiento del progreso. ¹
Backend	Node.js 22.14.0	Versión LTS con soporte a largo plazo, estabilidad garantizada.
Frontend	Angular CLI 21.2.2	Generación, servicio y compilación del proyecto Angular.
Base de datos	PostgreSQL 16 + pgAdmin 4	PostgreSQL como motor relacional; pgAdmin como interfaz gráfica de administración.
Diseño UI/UX	Figma	Creación de wireframes y prototipo de alta fidelidad.
Pruebas API	Postman	Prueba y documentación manual de los endpoints de la API REST durante el desarrollo.

¹ Enlace al repositorio GitHub del proyecto: <https://github.com/PilarLF/PlanifyApp/tree/main>

Categoría	Herramienta / Versión	Justificación
Validación accesibilidad	WAVE (WebAIM)	Análisis automatizado de accesibilidad web (WCAG 2.1 AA).

La instalación del entorno sigue el orden (1) Node.js y npm, (2) Angular CLI (`npm install -g @angular/cli`), (3) PostgreSQL que incluye pgAdmin. Los comandos y las instrucciones de instalación se recogen en el fichero README.md del repositorio del proyecto.

2.7.2. PostgreSQL.

Se ha utilizado **PostgreSQL** como base de datos relacional. Este modelo relacional se basa en organizar los datos en tablas (relaciones) compuestas por filas (registros únicos) y columnas (atributos de los registros), con identificadores primarios para cada fila (clave primaria) y claves foráneas que definen las relaciones entre tablas.

Este tipo de base de datos también es usado por otros programas como **Oracle** y **MySQL**. En este proyecto se ha usado una base de datos relacional porque el dominio con el que trabajamos (empleados, roles, turnos, puestos, etc) es inherentemente relacional. El uso de MongoDB, como base no relacional, por ejemplo, implicaría **perder integridad referencial**. Se ha descartado el uso de Oracle, por ejemplo, porque es un sistema con una licencia costosa e implica un entramado complejo propio de entornos de gran escala; sin embargo, PostgreSQL es, no solo más ligero, sino *open source* y gratuito. Además, se ha elegido frente a opciones como MySQL, porque este último ha presentado problemas de robustez.

En una aplicación como la nuestra, donde gestionamos horarios, fichajes e información personal que debe ser protegida y no manipulada, elecciones como el uso de Typescript y PostgreSQL son vitales. En resumen, PostgreSQL ofrece un sistema open source, gratuito, con buen soporte, integridad transaccional y cumplimiento del estándar SQL.

El esquema de la base de datos se recoge en el fichero 'planifiesquemas.sql'. Las tablas creadas son:

1. Tabla **usuarios**:

```
CREATE TABLE usuarios (
    id SERIAL PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    email VARCHAR(150) UNIQUE NOT NULL,
    password_hash VARCHAR(255) NOT NULL,
    role VARCHAR(20) NOT NULL CHECK (role IN ('ADMIN', 'EMPLOYEE')),
    position VARCHAR(100), -- camarero, cocinero, etc.
    created_at TIMESTAMP DEFAULT NOW()
);
```

Rol usa Check para evitar valores no válidos (que no sean admin o employee). Podemos ver PasswordHash, ya que no se guarda contraseñas en texto plano.

2. Tabla **horarios**:

```
CREATE TABLE horarios (
    id SERIAL PRIMARY KEY,
    employee_id INTEGER NOT NULL REFERENCES usuarios(id) ON DELETE CASCADE,
    start_time TIMESTAMP NOT NULL,
    end_time TIMESTAMP NOT NULL,
    CHECK (end_time > start_time)
);
```

Aquí se crea la tabla donde cada empleado tiene su turno, con su inicio y su final de jornada, no solapantes (esto se verifica en el backend) y bajo las reglas del convenio.

3. Tabla **fichajes**:

```
CREATE TABLE fichajes (
    id SERIAL PRIMARY KEY,
    employee_id INTEGER NOT NULL REFERENCES usuarios(id) ON DELETE CASCADE,
    schedule_id INTEGER NOT NULL REFERENCES horarios(id) ON DELETE CASCADE,
    clock_in TIMESTAMP NOT NULL,
    clock_out TIMESTAMP,
    total_minutes INTEGER,
    CHECK (clock_out IS NULL OR clock_out > clock_in)
);
```

Clock_out es NULL hasta que el empleado fiche la salida, tras fichar la entrada. Los empleados fichan su inicio y su final de la jornada laboral. Estos fichajes atienden a la tabla turnos, ya que no se puede fichar fuera del horario laboral del trabajador.

4. Tabla **fichajes_aud**:

```
CREATE TABLE fichajes_auditoria (
    id SERIAL PRIMARY KEY,
    fichaje_id INTEGER NOT NULL REFERENCES fichajes(id),
    employee_id INTEGER NOT NULL,
    schedule_id INTEGER NOT NULL,
    clock_in TIMESTAMP NOT NULL,
    clock_out TIMESTAMP,
    total_minutes INTEGER,
    changed_at TIMESTAMP DEFAULT NOW(),
    changed_by INTEGER REFERENCES usuarios(id),
    action VARCHAR(20) NOT NULL CHECK (action IN ('INSERT', 'UPDATE',
'DELETE'))
);
```

Aunque se dudó, finalmente se creó una **tabla fichajes_aud**. Es importante garantizar que los datos de los fichajes se mantienen almacenados e inmutables, a fin de garantizar la integridad laboral de los trabajadores y contar con los datos necesarios en caso de litigios legales. Recoge el id del empleado, el id de su horario, el valor del clock_in, clock_out y total_minutes. Cada vez que se crea o se manipula un fichaje queda reflejado en la tabla. Changed_by indica quién realizó tal cambio. Esto es útil porque desde el punto de vista del administrador puede consultar los fichajes de cada empleado y ver si ha fichado o no. Si algún día se le olvida fichar, el administrador puede modificar sus fichajes y añadirlos al boletín, pero siempre de forma transparente y reflejando estos cambios.

Para mejorar las consultas frecuentes, se crearon **índices**:

```
CREATE INDEX idx_usuarios_email ON usuarios(email);
CREATE INDEX idx_horarios_employee ON horarios(employee_id);
CREATE INDEX idx_horarios_start ON horarios(start_time);
CREATE INDEX idx_fichajes_employee ON fichajes(employee_id);
CREATE INDEX idx_fichajes_schedule ON fichajes(schedule_id);
```

Las restricciones horarias impuestas por el convenio laboral se aplicarán desde el backend, que hará las comprobaciones necesarias antes de insertar un nuevo turno.

2.7.3. *Desarrollo back end con Nojde.js, Express y TypeScript.*

En el lado del servidor podemos encontrar multitud de lenguajes de programación según diferentes paradigmas: PHP, Python, JAVA, Node.JS, etc. En contraste, del lado del cliente los lenguajes desarrollados han sido muy pocos y hoy en día existe claramente un único vencedor: JavaScript. No obstante, en los últimos años se ha impuesto un superconjunto de JavaScript, **TypeScript**. TypeScript permite desarrollar JavaScript, pero, a diferencia de JS, existe en tiempo de programación, ya que los navegadores web no saben interpretar TypeScript. Eso obliga a que el código escrito en TypeScript deba ser transformado a JavaScript para poder ser interpretado por los navegadores web (José Luis Chacón, 2025).

La utilidad de su uso, no obstante, radica en que TypeScript genera un **código más fiable**. JavaScript es un lenguaje de tipado dinámico, es decir, las variables pueden cambiar de tipo durante la ejecución del programa. Esto puede generar errores de tipo que solo serán detectados cuando el código esté corriendo. Si bien esto puede ser asumible en scripts pequeños, en aplicaciones grandes con múltiples desarrolladores implica la generación de bugs que se manifiestan durante la etapa de producción (Geeks for geeks, 2025; José Luis Chacón, 2025).

Combinar Node.js, Express y TypeScript para el backend facilitará el desarrollo y su integración con Angular, permitiendo la mantenibilidad y la escalabilidad. Que ambas capas estén creadas con TypeScript permite obtener una **coherencia tecnológica**. Además, se ha usado **Express** como framework siendo el más extendido en el ecosistema Node.js y contando con una amplia comunidad.

La estructura de carpetas del backend responde al patrón de **separación de responsabilidades** (*Separation of Concerns*), dividiendo el

código en capas con responsabilidades definidas, favoreciendo al mantenimiento y la legibilidad de este.

Tabla 13. Estructura de carpetas del backend.

Ruta	Descripción
backend/src/server.ts	Punto de entrada. Arranca el servidor HTTP en el puerto definido en .env.
backend/src/app.ts	Configura la aplicación Express: CORS, JSON parsing y montaje de rutas.
backend/src/config/db.ts	Crea y exporta el pool de conexiones a PostgreSQL mediante pg.Pool.
backend/src/middleware/auth.middleware.ts	Define authMiddleware (verificación JWT) y requireRole (control de acceso por rol).
backend/src/routes/auth.routes.ts	Endpoints de autenticación: POST /login, POST /register, GET /employees.
backend/src/routes/horarios.routes.ts	Endpoints de gestión de turnos: CRUD completo (solo ADMIN).
backend/src/routes/fichajes.routes.ts	Endpoints de fichaje: clock-in, clock-out, status, turno-actual, mis-turnos.
backend/src/controllers/auth.controller.ts	Lógica de negocio para autenticación y gestión de usuarios.
backend/src/controllers/horarios.controller.ts	Lógica de negocio para creación, lectura, actualización y eliminación de turnos.
backend/src/controllers/fichajes.controller.ts	Lógica de negocio para el registro de fichajes y consulta de turnos del empleado.
backend/tsconfig.json	Configuración de TypeScript: target ES2020, módulo CommonJS, rutas de entrada/salida.
backend/.env	Variables de entorno: credenciales de base de datos, puerto y secreto JWT.

Configuración de la aplicación y conexión a la base de datos

En **server.ts** se importa la instancia de Express configurada en **app.ts** e inicia la escucha en el puerto definido por la variable de entorno **PORT**. En **app.ts** se registran los middlewares globales, en orden: **cors()**, que habilita las peticiones necesarias para la comunicación con Angular; **express.json()**, que activa el parseo a JSON; y el montaje de las tres familias de rutas (**/api/auth**, **/horarios** y **/fichajes**). De esta forma, **app.ts** exporta la aplicación para que pueda ser importada en pruebas sin necesidad de levantar el servidor real.

Para llevar a cabo la **conexión con la base de datos** se crea el módulo **db.ts**, que instancia un **pg.Pool** con los parámetros leídos desde las variables de entorno (host, puerto, usuario, contraseña y nombre de la base de datos). Este pool permite crear un grupo de conexiones reutilizables en lugar de abrir y cerrar una conexión cada vez que se establece una consulta, mejorando el rendimiento de la aplicación.

Capa de middleware: autenticación y autorización

Un **middleware** es una función que se ejecuta **entre** la petición del cliente y la respuesta del servidor. Permite validar o modificar las peticiones antes de que lleguen al controlador. Es importante crearlo porque solo el rol de administrador podrá ver y **manipular** todos los fichajes y todos los horarios, pero no los empleados. Además, solo los empleados autenticados correctamente podrán acceder a sus fichajes y sus turnos. De esta forma, se **protege** el acceso a los datos y se restringe las acciones a ellos. Por ello, el middleware de la aplicación se compone de dos funciones:

- **authMiddleware**: verifica que el usuario está **autenticado** y que su token JWT es válido. Si todo es correcto, inserta los datos del usuario en `<req.user>`. Esto permite que el siguiente middleware, `requireAdmin`, sepa quién es el usuario y qué rol tiene.
- **requireAdmin**: permite la **autorización** basada en rol y se ejecuta después de `authMiddleware`, a fin de permitir acceder a ciertas rutas solo a los **administradores**. Así se protegen rutas como la gestión de empleados o el CRUD de horarios.

Juntos implementan un sistema de doble verificación y **seguridad por capas**.

Capa de rutas

La capa de rutas constituye el punto de entrada a la API REST. Se han organizado las rutas por dominios funcionales. Los tres módulos principales son:

- **auth.routes**: autenticación y gestión de usuarios. Contiene la ruta POST/login pública que incluye un `loginLimiter` para limitar los intentos fallidos de autenticación. GET/employees devuelve el listado de

empleados, requiere rol ADMIN y está protegida por ambos middlewares. POST/register permite crear nuevos usuarios.

- **fichajes.routes:** para el fichaje de los empleados. POST /clock-in y POST /clock-out exigen autenticación (el empleado solo puede fichar por sí mismo). También se ha decidido crear GET/status, que devuelve el estado actual del fichaje para mostrar si el empleado ha fichado o no en su dashboard. Con GET/turno-actual se obtiene el turno del empleado que abarca el momento presente; solo si hay un turno activo el empleado puede fichar.
- **horarios.routes:** gestiona los turnos (CRUD), solo para administradores. Las operaciones de lectura (GET /) y escritura (POST /, PUT /:id, DELETE /:id) están protegidas con authMiddleware y requireRole('ADMIN'), salvo la lectura del horario de un empleado concreto (GET /empleado/:id) que únicamente requiere autenticación.

Capa de controladores

Los controladores implementan la **lógica** de negocio. Reciben los parámetros de la petición validados por el middleware, realizan las consultas SQL necesarias y devuelven la respuesta. Los más importantes que contribuyen a perfilar el MVP son:

- **auth.controller.ts -login.** El proceso de login sigue estos pasos:
 - Se busca el usuario en la base de datos por su email.
 - Si no existe, se responde con error 404 ("no encontrado").
 - Se compara la contraseña introducida con el hash guardado usando bcrypt.
 - Si no coinciden, se responde con error 401 ("no autorizado").
 - Si todo es correcto, se genera un token JWT con el id y el role del usuario y se devuelve al frontend.
- **horarios.controller.ts – crear horario.** Las validaciones de horario han recaído en el backend. Este se encarga de comprobar:
 - **Solapamiento:** el nuevo turno no debe solaparse con otro turno del mismo empleado.

- **Descanso mínimo de 12 horas** entre el fin del último turno y el inicio del siguiente de un mismo empleado.
- Ningún empleado puede trabajar más de **11 días consecutivos**. Se lanza el error 400 en cualquiera de los tres casos anteriores si la validación falla.
- **fichajes.controller.ts -clockin**. Comprueba: el turno existe y pertenece al empleado, la hora actual está dentro del rango de turno y el empleado no tiene otro fichaje abierto sin cerrar.
- **fichajes.controller.ts -clockout**. Comprueba que el empleado tiene un fichaje abierto, registra la hora de salida y obtiene el total de minutos trabajados tras restar la hora de entrada y de salida.

2.7.4. Desarrollo front-end con angular

Se ha seleccionado un framework como Angular para desarrollar el frontend de la aplicación porque es una herramienta ampliamente vista en asignaturas cursadas en el máster. Esto pone de manifiesto la importancia de este framework en el campo de desarrollo web y permite aprovechar las competencias adquiridas. Además, se ajusta a ese proyecto por tres razones principales:

1. Es un framework “**opinionated**”: incluye soluciones integradas para peticiones HTTP, formularios y enrutamiento, por lo que no es necesario elegir entre múltiples librerías ni requiere instalar decenas de paquetes npm de terceros como ocurre con otros frameworks SPA como Vue o React (Stephen Fluin, 2017).
2. Usa **TypeScript** de forma nativa, generando un código robusto.
3. Genera un ecosistema cohesionado de componentes que favorece la **escalabilidad** y **modularidad**, y permite que equipos con múltiples desarrolladores tengan un código consistente y predecible sin tener que reinventar la rueda constantemente

De igual manera, cuenta con una gran comunidad y soporte, un hecho decisivo que puede facilitar el trabajo del desarrollador.

Para la construcción del frontend con Angular (v21) se ha seguido una arquitectura modular: cada módulo agrupa los componentes, servicios y rutas

relacionadas a su funcionalidad. En el directorio principal, `src/app`, la organización de ficheros es:

Tabla 14. Estructura de carpetas del frontend.

Ruta	Descripción
<code>src/app/app.ts</code>	Componente raíz. Registra el locale español (DatePipe en español) y renderiza el router-outlet principal.
<code>src/app/app.config.ts</code>	Configuración de la aplicación: proveedores de rutas, HttpClient con el interceptor de token.
<code>src/app/app.routes.ts</code>	Definición de rutas lazy-loaded: auth, admin y employee con sus respectivos módulos y componentes.
<code>src/app/auth/</code>	Módulo de autenticación: login, guard y interceptor HTTP.
<code>src/app/admin/</code>	Módulo del panel de administración: dashboard y gestión de horarios.
<code>src/app/employee/</code>	Módulo del portal de empleado: layout, navbar, dashboard y servicio de fichajes.
<code>src/app/user/user.ts</code>	Servicio para obtener el listado de empleados (consumido por el panel de administración).
<code>src/app/models/user.model.ts</code>	Interfaz TypeScript que define la estructura del objeto User.
<code>src/styles/</code>	Estilos globales y variables SCSS compartidas por todos los módulos.

Por una parte, se han creado **servicios**. En Angular, los servicios son clases reutilizables que centralizan la lógica de acceso de los datos. Esto es útil porque, en la práctica, lo habitual es que haya más de dos componentes que quieran acceder a los mismos datos y realizar operaciones similares. El servicio más complejo que se ha creado es **auth.service**, encargado de realizar las peticiones HTTP y gestionar el estado de sesión del usuario. Utiliza un BehaviorSubject, que es un tipo especial de variable reactiva: cuando su valor cambia, todos los componentes suscritos se actualizan automáticamente.

Otros elementos de Angular son los interceptores y los guards. En Angular, los **interceptores** son funciones que implementan la lógica de la interfaz `HttpInterceptor`, a fin de interpretar y manipular solicitudes y respuestas HTTP (Cristian Arieta, 2025a). El único interceptor global creado para PlanifyApp ha sido **tokenInterceptor** dentro del módulo `/auth`. La función de los **guards** es la de proteger las rutas, activándolas y desactivándolas. Esto ayuda a manejar la navegación de los usuarios por la aplicación de forma eficiente (Cristian Arieta,

2025b). Para PlanifyApp se ha creado dos guards: uno para el módulo /auth (auth.guard.ts), que impide el acceso a cualquier ruta privada si el usuario no está autenticado tras comprobar el token a través de auth.service; y otro para el módulo /admin (admin.guard.ts), que permite el acceso a rutas exclusivas de los administradores.

En la misma línea, otro componente principal de Angular son los **módulos**. Constituyen las unidades funcionales cohesivas, contenedores que agrupan componentes y encapsulan funcionalidades en bloques de código independientes (Alan Sastre, 2024).

Módulo de autenticación (auth/)

Agrupar los elementos encargados de gestionar el acceso a la aplicación.

- **Login (login/login.ts):** componente standalone que renderiza el formulario de acceso mediante ReactiveFormsModule. Utiliza FormBuilder para construir un FormGroup con validaciones de campo requerido y formato de email. Al enviar el formulario, llama a AuthService.login(), almacena el token JWT y el rol en localStorage y redirige al usuario al dashboard correspondiente (/admin/dashboard o /employee/dashboard) según el valor del campo role devuelto por el backend.
- **AuthService (auth.service.ts):** servicio inyectable que encapsula todas las peticiones HTTP relacionadas con la sesión: login(), logout() (limpia localStorage), isAuthenticated() (comprueba la existencia del token) y register(). Su uso centralizado evita duplicar las URLs del backend y facilita el mantenimiento.
- **authGuard (auth-guard.ts):** guardia funcional de tipo CanActivateFn. Comprueba la presencia del token en localStorage; si existe, devuelve true y permite el acceso a la ruta; si no, redirige al login y devuelve false. Se aplica sobre las rutas protegidas de los módulos admin y employee.
- **tokenInterceptor (token-interceptor.ts):** interceptor funcional de tipo HttpInterceptorFn, registrado globalmente en appConfig mediante

provideHttpClient(withInterceptors([tokenInterceptor])). Clona cada petición HTTP saliente y añade el encabezado Authorization: Bearer <token>, evitando tener que incluirlo manualmente en cada llamada de los servicios

Módulo de administración (admin/)

El módulo admin agrupa las funcionalidades del rol administrador: la visualización de todos los turnos y la creación, modificación y eliminación de los mismos.

- **AdminDashboard (admin/dashboard/dashboard.ts):** componente principal del panel de administración. En su ciclo ngOnInit carga en paralelo el listado de empleados (UserService.getEmployees()) y el listado completo de horarios (Horarios.getHorarios()). Expone métodos para crear un nuevo turno (crearHorario()), editar uno existente (editar()) y guardarEdicion()) y eliminar (borrar()). La edición se materializa mediante un formulario inline que aparece al seleccionar el turno, vinculado a la variable editando mediante ngModel.
- **Horarios (admin/horarios.ts):** servicio Angular que centraliza las llamadas HTTP a /api/horarios. Implementa los métodos getHorarios(), getHorariosEmpleado(id), createHorario(data), updateHorario(id, data) y deleteHorario(id), devolviendo los Observables correspondientes para que los componentes se suscriban y gestionen la respuesta.
- **UserService (user/user.ts):** servicio que expone getEmployees(), el cual realiza un GET a /api/auth/employees incluyendo el token de autorización. Es utilizado por el AdminDashboard para poblar el selector de empleados en el formulario de creación de turnos.
- **AdminRoutingModule (admin-routing.module.ts):** módulo de enrutamiento del área de administración. Define la ruta dashboard, que activa el componente AdminDashboard y aplica el authGuard.

El módulo employee implementa el portal personal del trabajador, que integra el fichaje, la visualización de turnos y un calendario mensual interactivo.

- **EmployeeLayoutComponent (layout/employee-layout.ts):** componente contenedor que actúa como shell del área de empleado. Renderiza la barra de navegación (EmployeeNavbar) y el router-outlet secundario que carga los componentes hijos. Esta separación permite que la barra de navegación sea persistente mientras el contenido central varía según la ruta activa.
- **EmployeeNavbar (navbar/employee-navbar.ts):** componente de navegación que muestra el logo de PlanifyApp y los enlaces de la sección de empleado. El método logout() elimina el token de localStorage y redirige al login. Importa RouterModule para que los enlaces routerLink funcionen correctamente.
- **Dashboard (employee/dashboard/dashboard.ts):** componente central del portal de empleado. Implementa la interfaz OnInit e inyecta FichajesService y ChangeDetectorRef. En la inicialización lanza tres peticiones en paralelo: loadTurnoActual() (determina el turno cuyo rango horario incluye el momento actual), loadStatus() (recupera el estado de fichaje: si el empleado está o no fichado, y las últimas marcas temporales) y loadMisTurnos() (obtiene la lista completa de turnos asignados). Los métodos clockIn() y clockOut() delegan en el servicio el registro de las marcas temporales. La función estaDentroDelTurno() calcula si el momento presente está dentro del rango del turno actual, información usada para mostrar la insignia de estado en la tarjeta de turno. Los turnos obtenidos se mapean al formato de eventos de FullCalendar (title, start, end, color) y se asignan a la propiedad events de calendarOptions.
- **FichajesService (fichajes/fichajes.ts):** servicio que encapsula todas las llamadas HTTP al endpoint /api/fichajes. Incluye un método privado getHeaders() que recupera el token de localStorage y lo incluye en el encabezado Authorization de cada petición. Expone: getStatus(), getTurnoActual(), clockIn(schedule_id), clockOut() y getMisTurnos(). El

uso de este servicio centraliza la lógica de comunicación con el backend y desacopla los componentes de los detalles de la API.

PlanifyApp no utiliza una librería de estado centralizada como NgRx o Akita, optando en su lugar por un patrón de servicios con BehaviorSubject de RxJS para la información que necesita ser compartida entre componentes. Para el resto de datos (horarios, fichajes), cada componente invoca directamente al servicio correspondiente al inicializarse (ngOnInit), siguiendo un flujo reactivo simple que resulta suficiente para la escala actual del proyecto.

2.7.5. Test de validación

Para garantizar la funcionalidad y calidad del backend, se han implementado pruebas unitarias y de integración. Las pruebas unitarias son pruebas automáticas que validan el comportamiento de una unidad de código y permiten asegurar que funcionan según lo esperado (Nahum Ramírez, 2024).

Aunque Angular permite la creación automática de estas pruebas, para el backend se han creado manualmente. Así, los tests unitarios cubren los middlewares authMiddleware y requireAdmin, poniendo a prueba los casos de autenticación inválida, revocación de tokens, acceso restringido por rol y el funcionamiento correcto de los endpoints. Además, se desarrollaron tests de integración que validan el flujo completo de uso del sistema: autenticación del usuario, consulta de turnos, registro de fichajes y validación de reglas del convenio laboral.

En cuanto al frontend, se realizó una validación de accesibilidad real sobre la aplicación desplegada en Netlify utilizando WAVE y Lighthouse. Estas herramientas permitieron detectar problemas de contraste, etiquetas ARIA ausentes y mejoras en navegación por teclado, garantizando el cumplimiento de las pautas WCAG 2.1 AA.

Para realizar los test unitarios se ha usado Jest, un framework de testing JavaScript creado por Facebook. Es el más utilizado y uno de los mejor

valorados por su simplicidad de implementación, documentación y comunidad activa (Yuri Kan, 2026).

requireAdmin.spec.ts comprueba:

- Que req.user exista (si no, error 401).
- Que req.user.role sea admin (si no, error 403).
- Si todo está bien, llama a next().

authMiddleware.spec.ts comprueba:

- Que haya token (error 401)
- Que el token sea válido (403)
- Que el usuario exista en la base de datos (401)
- Que el token coincida (401)
- Si todo está bien, llama a next().

Ambos se han construido importando el middleware correspondiente y creando objetos falsos como mockReq o mockRes, que simulan req y res, respectivamente. Se han escrito tantos casos de prueba como caminos del middleware hay.

Para los test de los controladores se ha comprobado el funcionamiento de los endpoints: recepción de datos, llamada al servicio, respuesta y manejo de errores. Se ha creado un único test con tres bloques describe(), uno por cada controlador.

auth.controller.spec.ts comprueba:

- Login correcto
- Login con usuario inexistente
- Login con contraseña inválida
- Login con errores de validación
- Error interno del servidor

2.7.6. *Despliegue en servidor público.*

Una vez desarrollada la aplicación en entorno local se ha desplegado en un servidor público accesible a través de internet a fin de evaluar su funcionamiento real fuera del repositorio local. La arquitectura de despliegue reproduce la separación de capas propia del diseño de la aplicación: frontend, backend y base de datos se alojan en plataformas independientes y especializadas, comunicadas entre sí a través de sus respectivas URLs públicas. Para ello se ha usado **Render** y **Netlify**, ya que ambas son **gratuitas** y permiten el despliegue de la app directamente desde el repositorio **Git**.

Backend y base de datos: Render

Se ha seleccionado Render para desplegar el backend porque es sencillo, gratuito y ofrece despliegue automático. Desde la página web, nos creamos nuestra cuenta y se importa el proyecto desde Github. Tras configurar las variables de entorno, Render se encarga de hacer el build y lanzar el backend en un contenedor gestionado.

Aunque la capa gratuita no da acceso a todas las características de la aplicación, se ha elegido frente a otras alternativas como Railway porque ofrece una documentación más extensa para proyectos Node.js/Express y, al igual que Netlify, soporta despliegue automático continuo ante cambios en el repositorio Github. Sin embargo, Render presenta cold starts cuando el servicio permanece inactivo durante 15 minutos, lo que puede provocar una latencia inicial en la primera petición tras un periodo sin actividad. Aunque para el contexto académico del TFM sea suficiente, debería considerarse este aspecto si quisiese trasladarse a un entorno productivo real.

Por otra parte, se ha elegido Render porque proporciona alojamiento para la base de datos PostgreSQL. De esta forma, se simplifica la gestión de credenciales y configuración de entorno y la conectividad con el backend. Así, esta solución integrada reduce la superficie de configuración.

Frontend: Netlify

El despliegue del frontend se ha realizado en Netlify, una plataforma de alojamiento estático que, igual que Render, ofrece integración nativa con repositorios Git y despliegue continuo. Para la implementación, se ha creado un fichero `netlify.toml` donde se recoge las reglas de redirección y permite que Angular pueda gestionar la navegación en el lado del cliente con una configuración mínima. Para los requisitos del proyecto, la capa gratuita de Netlify es suficiente.

3. Resultados

3.1. Estudio de usabilidad

3.1.1. Pruebas de evaluación heurística

La evaluación heurística se realizó sobre el prototipo de alta fidelidad de Figma. Se realizó con dos evaluadores, ambos con conocimiento de Ingeniería de Software e Ingeniería Multimedia, respectivamente. Siguiendo la escala propuesta por Nielsen, puntuaron del 0 al 4 (donde 0 indica ausencia de problema y 4 presencia de problema crítico) los heurísticos recogidos en la siguiente tabla:

Tabla 15. Resultados de la prueba de evaluación heurística.

#	Heurístico	Severidad 0-4	Observaciones
H1	Visibilidad del estado del sistema	1,0	El indicador de estado de fichaje es visible, pero su posición en pantalla puede pasar desapercibida en dispositivos de pantalla pequeña.
H2	Correspondencia con el mundo real	0,0	La terminología utilizada es adecuada al contexto laboral. Los iconos resultan reconocibles e intuitivos para el público objetivo.
H3	Control y libertad del usuario	1,1	Las acciones destructivas incluyen diálogo de confirmación. No obstante, en algunos flujos de navegación no se ofrece una opción explícita de "cancelar" para volver al estado anterior.
H4	Consistencia y estándares	0,0	El sistema de diseño es coherente en toda la aplicación. Los componentes respetan el estándar Material Design de manera uniforme.
H5	Prevención de errores	2,0	La validación de solapamiento de turnos funciona correctamente, pero el mensaje de advertencia sobre el descanso mínimo entre jornadas no se muestra de forma suficientemente prominente antes de que el usuario intente guardar.
H6	Reconocer antes que recordar	1,1	El dashboard muestra los turnos del día correctamente. Sin embargo, para los administradores la información de resumen es escasa y obliga a navegar a otras secciones para obtener una visión completa.
H7	Flexibilidad y eficiencia de uso	0,1	La arquitectura de navegación plana cumple el criterio de máximo 2–3 clics para las acciones principales. Adecuado para el alcance del MVP.

#	Heurístico	Severidad 0-4	Observaciones
H8	Estética y diseño minimalista	0,0	La interfaz es limpia y libre de elementos innecesarios. La jerarquía visual es clara y facilita la lectura.
H9	Ayuda a reconocer y recuperarse de errores	1,0	Los mensajes de error son descriptivos. En algunos casos, sin embargo, el mensaje aparece en una posición alejada del campo que lo origina, lo que puede dificultar su asociación.
H10	Ayuda y documentación	2,2	Los tooltips están implementados en los elementos principales, pero la sección de ayuda contextual no está disponible en todos los módulos del prototipo.

3.1.2. Pruebas con usuarios reales

Los 5 participantes seleccionados para la prueba han sido escogidos sin perfil técnico, ya deben ser representativos de los usuarios que van a usar esta aplicación. Todas las sesiones se han llevado a cabo de forma individual y se realizaron con el prototipo de alta fidelidad con interacción de Figma.

Tabla 16. Resultados de la prueba de usabilidad con usuarios.

ID	Tarea	Perfil	Tasa de éxito	Tiempo medio	Incidencias
T-01	Iniciar sesión	Ambos	5/5 (100%)	18 seg.	Ninguna.
T-02	Consultar turnos semanales	Empleado	5/5 (100%)	42 seg.	Un usuario buscó inicialmente la información en el menú de perfil.
T-03	Fichar entrada	Empleado	5/5 (100%)	14 seg.	Ninguna.
T-04	Crear turno para un empleado	Administrador	3/3 (100%)	2 min. 10 seg.	Un administrador tardó en localizar el selector de empleado dentro del formulario.
T-05	Consultar fichajes del día	Administrador	3/3 (100%)	1 min. 05 seg.	Dos usuarios navegaron inicialmente a la sección incorrecta antes de localizar el registro.
T-06	Modificar turno existente	Administrador	2/3 (67%)	2 min. 48 seg.	Un usuario no encontró la opción de edición al no identificar el icono de lápiz como acción interactiva.

3.2. Desarrollo de la aplicación

El servidor Node.js/Express está completamente operativo en local. Los doce endpoints definidos en la Tabla 7 han sido verificados mediante Postman, comprobando los códigos de estado HTTP esperados tanto en los flujos principales como en los alternativos (credenciales incorrectas, solapamiento de turnos, fichaje fuera de horario, token expirado, etc.).

Las tres validaciones laborales implementadas en el controlador de creación de horarios funcionan correctamente: el sistema rechaza turnos con la hora de fin anterior a la de inicio, detecta solapamientos con otros turnos del mismo empleado y verifica el cumplimiento del descanso mínimo de 12 horas entre jornadas. Cada caso de error devuelve un mensaje descriptivo que el frontend puede mostrar directamente al usuario.

La aplicación Angular compila sin errores y está operativa en el entorno de desarrollo local (ng serve). Las funcionalidades implementadas y verificadas son:

- Autenticación completa: el formulario de login valida los campos en tiempo real, realiza la petición al backend, almacena el token JWT en localStorage y redirige al dashboard correspondiente según el rol.
- Guard de rutas: el authGuard protege correctamente las rutas /admin/* y /employee/*, redirigiendo al login a los usuarios no autenticados.
- Interceptor de token: el tokenInterceptor inyecta automáticamente el encabezado de autorización en todas las peticiones HTTP, eliminando la necesidad de gestionarlo manualmente en cada servicio.
- Panel de administración: el formulario de creación de turnos carga dinámicamente el listado de empleados, permite seleccionar el empleado y las fechas de inicio y fin, y muestra los mensajes de éxito o error devueltos por el backend.
- Portal de empleado — estado de fichaje: la tarjeta de estado muestra en tiempo real si el empleado está fichado o no, con el indicador visual animado (punto verde pulsante cuando está activo) y la última marca temporal registrada.

- Portal de empleado — turno actual: la tarjeta de turno muestra las horas de inicio y fin del turno activo en el momento actual, con la insignia de estado (En turno / Fuera de turno) calculada en el cliente.
- Portal de empleado — fichaje: los botones Fichar entrada y Fichar salida realizan correctamente las peticiones al backend e incluyen el `schedule_id` del turno actual en la petición de clock-in.
- Portal de empleado — lista de turnos: se renderiza la lista completa de turnos asignados al empleado, con la fecha en formato de bloque visual (número de día y mes abreviado) y el rango horario.
- Calendario FullCalendar: los turnos del empleado se representan como eventos en el calendario mensual, con el color corporativo y las horas de inicio y fin como título del evento.

3.2.1. Test unitarios

```
PS C:\Users\plope\OneDrive\tfm\planify-app> npm test

> test
> jest

PASS backend/tests/requireAdmin.spec.ts
PASS backend/tests/authMiddleware.spec.ts
  • Console

Node.js v22.14.0

Test Suites: 2 passed, 2 total
Tests:      8 passed, 8 total
Snapshots:  0 total
Time:       4.268 s
Ran all test suites.
```

3.2.2. Despliegue en servidor público

Como resultado del proceso de despliegue descrito en el apartado de Material y Métodos, PlanifyApp se encuentra accesible públicamente en la URL <https://planifyapphrr.netlify.app/>. Se ha verificado el correcto funcionamiento del flujo completo de login, registro y acceso al dashboard redirigido según rol y sus competencias.

3.2.3. Aspectos pendientes

En el momento de redacción de esta memoria, se tienen en cuenta diversos aspectos a mejorar e implementar de cara a la entrega final:

- Mejora del UX. Para esta entrega se ha priorizado la implementación de las funcionalidades básicas de la aplicación, por lo que se ha dejado de lado aspectos estéticos. Problemas surgidos durante el desarrollo local de la aplicación y el despliegue en el servidor público ha retrasado las tareas correspondientes a la accesibilidad de la web y la implementación de un diseño responsive.
- Completar el panel de administrador: vistas de turnos de los empleados, listado de los empleados, etc.
- Redacción de la documentación técnica de cara a la entrega final (Readme.md, código fuente limpio y ordenado, etc).
- Test unitarios para los controladores y test de integración para analizar el flujo completo.

4. Bibliografía

- 5 pasos para realizar un estudio de viabilidad de un proyecto.* (2024, May 24). Cegos.
- Adrià Turneu Puig. (2024, January 18). *Benchmarking: Optimizando el rendimiento empresarial.* Blogs UOC.
- Alan Sastre. (2024, August 27). *Desarrollo de módulos Angular.* <https://certidevs.com/tutorial-angular-modulos>
- Alejandro Moreno Montero. (2025, October 6). *WCAG 2.2: Qué son y cómo aplicarlas.* Discapnet. <https://www.discapnet.es/accesibilidad/marketing-inclusivo/accesibilidad-web/wcag-22-que-son-y-como-aplicarlas>
- Capterra. (2025, January 4). *Pricing for Sesame HR.* Capterra. https://www.capterra.com/p/143413/SesameHR/#Capterra__6645914
- Carlos Vicente. (2024). *Análisis heurístico: guía completa con las heurísticas de usabilidad de Nielsen.* . CarlosVicente. <https://carlosvicente.es/blog/analisis-heuristico-guia-completa-con-las-heurísticas-de-usabilidad-de-nielsen/>
- Clements, P. C. (1999). *Software Architecture Software Architecture in in Practice Practice Software Engineering Institute.*
- Cristian Arieta. (2025a, March 10). *Interceptors en Angular: ¿para qué se utilizan, sus funciones y cómo implementarlos?* https://dev.to/cristian_arieta_7df932e5f/interceptors-en-angular-para-que-se-utilizan-sus-funciones-y-como-implementarlos-2mo8
- Cristian Arieta. (2025b, March 25). *Guards en Angular: protegiendo nuestras rutas.* https://dev.to/cristian_arieta_7df932e5f/guards-en-angular-protegiendo-nuestras-rutas-1677
- Factorial. (2026). *Página web de la empresa.* <https://factorial.es/>
- Geeks for geeks. (2025, August 5). *8 Reasons why you should pick TypeScript over JavaScript.* Geeks for Geeks. <https://www.geeksforgeeks.org/blogs/8-reasons-why-you-should-pick-typescript-over-javascript/>
- General de Industria PYME, D. (2030). *Marco Estratégico en política de PYME 2030.*

GetApp. (2026, March). *Sesame HR - 2026 Pricing, Features, Reviews & Alternatives*. GetApp. <https://www.getapp.com/hr-employee-management-software/a/sesame-time/>

Guía completa de diagramas de casos de uso UML. (2025, March 28). Go UML. <https://www.go-uml.com/comprehensive-guide-to-uml-use-case-diagrams/>

Holded. (2026). *Página web de la empresa*. 2026. <https://www.holded.com/es>

Jakob Nielsen. (1994, April 24). *10 Usability Heuristics for User Interface Design*. Norman Nielsen Group. <https://www.nngroup.com/articles/ten-usability-heuristics/>

José Luis Chacón. (2025). *TypeScript: qué es, diferencias con JavaScript y por qué aprenderlo*. Profile. <https://profile.es/blog/que-es-typescript-vs-javascript/>

Lakatos, G., Vámos, B. Z., Aupek, I., & Andó, M. (2025). Organization of the Optimal Shift Start in an Automotive Environment. *Computation*, 13(8). <https://doi.org/10.3390/computation13080181>

Nahum Ramírez. (2024, May 28). *Guía completa sobre pruebas unitarias*. https://dev.to/nerm_frontend/guia-completa-sobre-pruebas-unitarias-en-angular-16-4djl

Nueva normativa de accesibilidad web: ¿qué exige la nueva ley y cómo te afecta? (2025, July 9). Matteria. <https://materia.si/blog/normativa-accesibilidad-web-que-exige-la-nueva-ley-y-como-te-afecta/#:~:text=Requisitos%20t%C3%A9cnicos%20principales,de%20asistencia%20actuales%20y%20futuras.>

Olga Carreras. (2012, March 15). *Estándares formales de usabilidad y su aplicación práctica en una evaluación heurística*. Usable y Accesible. <https://olgacarreras.blogspot.com/2012/03/estandares-formales-de-usabilidad-y-su.html>

Paulo Cesar Vicente Da Luz. (2023, August 31). *Desvelando el Mito de los 5 usuarios en las Pruebas de Diseño: más allá de la cantidad*. LinkedIn. <https://www.linkedin.com/pulse/desvelando-el-mito-de-los-5-usuarios-en-las-pruebas-vicente-da-luz/>

Sesame HR. (2026). *Página web de la empresa*. <https://www.sesamehr.fr/>

Stephen Fluin. (2017, December 25). *Why Developers and Companies Choose Angular*. Medium. <https://medium.com/angular-japan-user-group/why->

developers-and-companies-choose-angular-

4c9ba6098e1c#:~:text=Angular%20is%20Scaled,number%20of%20develo
pers%20to%20collaborate

Susan Snipes. (2026, February 18). *Factorial HR review*. Remote People.

<https://remotepeople.com/providers/factorial-hr-review/>

Todo sobre la nueva ley de control horario de los trabajadores. (2025, December 31). Controllaboral.

TramitApp. (2026). *Página web de la empresa*. <https://www.tramitapp.com/>

User persona. (2025). Miro. <https://miro.com/es/persona/que-es-user-persona/>

Ying, K. C., Pourhejazy, P., & Lin, S. C. (2025). An exploration of the shift work consideration in production scheduling. *Journal of Computational Science*, 92. <https://doi.org/10.1016/j.jocs.2025.102724>

Yuri Kan. (2026, March 18). *Jest Tutorial: Guía completa de Unit Testing en JavaScript*. <https://yrkan.com/es/blog/jest-testing-tutorial/>